

Traceable Threshold Batch Encryption with Applications to Enhancing Mempool Privacy

Anirban Chakrabarti
anirban2chakrabarti@gmail.com
Indian Institute of Science
Bangalore, India

Monosij Maitra
monosij@cse.iitkgp.ac.in
IIT Kharagpur
Kharagpur, India

Arup Mondal
arup.mondal_phd19@ashoka.edu.in
Ashoka University
Sonapat, India

ABSTRACT

Building robust ways to outwit MEV strategies by network validators in cryptocurrency applications is garnering rapid interests now. Encrypting mempools with (varieties of) threshold encryption systems is seen as one of the leading solutions to this problem. Particularly, the notion of *Batched Threshold Encryption* (BTE) proposed in multiple recent papers by Choudhuri et al. [USENIX 2024, 2025] and Agarwal et al. [CRYPTO 2025] gives promising approaches to build scalable encrypted mempools. However, the existing BTE does not address collusion among (potentially *malicious*) validators (leaking confidential information).

We *initiate* the study of *Traceable Batched Threshold Encryption* (T-BTE) with the aim to enhance privacy of mempools. T-BTE integrates accountability (via *traceability*) into BTE, enabling not only thresholdized decryption (via a quorum of some t parties), but additionally with the *identification* of malicious validators. We present the *first* definitions and a construction for T-BTE systems satisfying CCA security. In more detail, we build T-BTE from the notion of a *Bipartite Batched Threshold Encryption* (Bi-BTE) – a primitive we *first* conceptualize and then build it from bilinear pairings and a one-time signature, and prove its security in the GGM. Our final T-BTE scheme builds upon Bi-BTE and a fingerprinting code (Boneh et al., DRM 2010). We implement and benchmark Bi-BTE and fingerprinting code in Rust to instantiate our T-BTE showcasing its practical potential for encrypted mempools. To the best of our knowledge, our work provides the *first* implementation of a fingerprinting code.

CCS CONCEPTS

• **Security and privacy** → **Public key encryption; Privacy-preserving protocols.**

KEYWORDS

Threshold Encryption, Batched Threshold Encryption, Encrypted Mempool, Mempool Privacy, Traceability.

ACM Reference Format:

Anirban Chakrabarti, Monosij Maitra, and Arup Mondal. 2026. Traceable Threshold Batch Encryption with Applications to Enhancing Mempool

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

ASIA CCS '26, June 1–5, 2026, Bangalore, India

© 2026 Association for Computing Machinery.

ACM ISBN 979-8-4007-2356-8/26/06...\$15.00

<https://doi.org/10.1145/3779208.3805977>

Privacy. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26)*, June 1–5, 2026, Bangalore, India. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3779208.3805977>

1 Introduction

A Batched Threshold Encryption (BTE) [2, 16, 18, 25, 26] scheme enables a group of servers to jointly decrypt (in a threshold fashion) any chosen subset/batch of ciphertexts from a large public pool, while maintaining the privacy of remaining ciphertexts. The defining feature of BTE is scalability: the total server-to-server communication required for decryption can grow only sublinearly with (or ideally, be completely independent of) the batch size. This ability to selectively and efficiently decrypt *only* a batch of ciphertexts is useful in many contexts, such as in the following: *Time-locked* (or *event-conditioned*) releases: Servers can jointly decrypt only after a prescribed condition – such as a timestamp, or a price trigger – has been met [1, 30]; *Fair multi-party computation*: Outputs of many concurrent MPC sessions can be released fairly and simultaneously once a threshold of participants consent [2, 27]; *Privacy-preserving auctions/derivatives*: only winning bids or exercised options are revealed, keeping all other bids private [3, 37].

While such usecases illustrate BTE's generality, one of its most immediate and crucial application today lies in *mempool privacy* and *MEV mitigation* in cryptocurrency and DeFi applications.

Mempools and MEV. In existing cryptocurrencies, users send transactions to a public queue (the mempool) before they are finalized in blocks (in its blockchain). These transactions are stored in plain, so miners or validators who are in control of block production can frontrun, reorder, replicate or censor such transactions for profit, at the expense of ordinary users. This phenomenon is called “Maximal Extractable Value” (MEV) [5, 28]. MEV undermines decentralization [28], and has become one of the most persistent structural issues in decentralized finance, with hundreds of millions of USD extracted [38] and massive lawsuits underway [36].

Recent works have proposed to reduce MEV through *encrypted mempools* [6, 34]. In encrypted mempools, users encrypt their transactions under a public key, whose associated secret key is shared within the validators. Once a block is finalized, the validators jointly decrypt only the transactions included in that block, while all unconfirmed transactions remain private. BTE naturally fits this setting: It enables threshold decryption of arbitrary subsets of ciphertexts (i.e., transactions included in each block) with communication that is *independent of the batch size* (i.e., the block size), while preserving semantic security for all pending transactions outside the batch (i.e., mempool transactions not included in the blocks).

Despite many recent works (see Sec. 1.4), the existing BTE schemes fails to provide complete security in most natural applications (including the one discussed above). One crucial issue is the lack of *accountability* in BTE. Integrating accountability in threshold cryptography has generally been a persistently difficult problem to tackle when a t -sized quorum (having the power to decrypt) act *maliciously*. More specifically, any set of at least t (malicious) parties has enough information to build the entire decryption key (including the share of *every other* party) – they can jointly build a *decoder* (hardwired with the full decryption key) that decrypt any input ciphertext to output a message, and sell it to an adversary.¹ Without accountability, identifying parties with such out-of-band activity becomes almost impossible to detect. Obtaining stronger (particularly CCA) security is another important requirement in such settings. CCA-security is crucial in all practical deployments of threshold encryption. This also holds in our context of encrypted mempools and has also been discussed in recent works [8, 18]. In particular, averting mauled (or rogue) ciphertext attacks and copy attacks are essential to really push such primitives towards a potential deployment. We refer to [18, Section 2] and [8, Section 1.1] for more details. Below, we define our contributions in this work.

1.1 Our Contributions

We formally integrate tracing into BTE and present practical, provably secure schemes, with a prototype implementation and requisite benchmarks. Our main contributions are as follows:

- (1) Our work *initiates* the study of *Traceable Batched Threshold Encryption* (T-BTE) aiming to enhance mempool privacy. To this end, we also *establish* the notion of *Bipartite Batched Threshold Encryption* (Bi-BTE) – a building block to obtain T-BTE. Below are more details.
- (2) We formalize Bi-BTE definitions (with CCA security) and build a CCA-secure Bi-BTE scheme from bilinear pairings that we prove secure in the generic group model (GGM).
- (3) We then formalize T-BTE definitions (with CCA security) and instantiate a CCA-secure T-BTE scheme using our Bi-BTE, a one-time signature and a fingerprinting code as building blocks.
- (4) We provide an open-source prototype implementation of fingerprinting codes [12] in Rust. To the best of our knowledge, our work provides the *first* implementation of a fingerprinting code. We use this codebase to finally implement a prototype of our T-BTE scheme in Rust to demonstrate its concrete efficiency in encrypted mempools. For $n = 64$ parties with a threshold $t = \lfloor n/2 \rfloor = 32$, batch size $B = 128$, encryption, partial batch decryption, partial batch decryption combine and batch decryption take 1.07s, 6.47ms, 1.17ms and 389.40ms, respectively, with a ciphertext size of 544Byte. For $n = 16$ with $t = 8$ and $B = \{8, 16\}$, tracing algorithms takes 339.85s and 681.57s, respectively.

Our techniques are inspired from the recent works by Agarwal et al. [2], Bhattacharya et al. [8] and Boneh et al. [13]. Our work solves an important open problem in the context of all the above results. We believe our contributions are a crucial stepping stone pushing the state of art to integrate accountability into mempools (and all other relevant applications).

¹An adversary could also somehow steal some secret key share(s) and obtain the rest of them through out-of-band collusion (see Ronin network hack [7, 9], for instance.)

1.2 Tracing Model in Practice & Our Setting

We consider a (t, n) threshold structure, where any t parties can jointly decrypt a batch of B ciphertexts. A collusion of at least t parties may create a decryption (or decoder) box D , that can leak even partial plaintext information. This is formalized via distinguishing advantage in [8, 13]. Thus, to enable traceability, the systems integrates a tracing algorithm that, with black-box access to D (and possibly a private tracing key), identifies at least one colluder.

Our tracing model focuses on the case where a t -sized malicious quorum “sells the decoder” rather than “selling decoder-as-a-service”. In encrypted mempools, this is more incentivized for an adversary to own the decoder directly, as otherwise the malicious members get frontrunning opportunities over the adversary in extracting the MEV. We refer to [13, Section 1.1] for more details.

1.3 Technical Overview

We provide a detailed overview of our techniques now. We first briefly discuss fingerprinting codes [12] and bipartite threshold encryption [13] as our construction builds on these fundamental tools to integrate accountability through tracing.

Fingerprinting Code. A fingerprinting code (formally defined in Appendix A.5) consists of two algorithms: (Gen, Trace). The algorithm Gen outputs a code (a set of codewords) along with a secret tracing key tk . The algorithm Trace takes as input a pirate word and the tracing key tk , and outputs the identity of at least one contributing player. Consider a binary fingerprinting code that contains n codewords (each denoted by $\bar{w} \in \{0, 1\}^L$) of length L , assigned to n players. Here, n and L are input parameters, where $n, L \in \mathbb{N}$. This code can be represented as an $n \times L$ binary matrix. Suppose a pirate corrupts c (out of n) players to obtain their respective codewords, denoted by the set $\{\bar{w}^{(1)}, \dots, \bar{w}^{(c)}\}$. This forms a $c \times L$ sub-matrix of the original code corresponding to the c rows of corrupted codewords. The pirate then constructs a forged word $\bar{w} \in \{0, 1\}^L$ of its choice under the constraint that in any coordinate where *all* $\bar{w}^{(1)}, \dots, \bar{w}^{(c)}$ are zero, the pirate word $\bar{w}$ must also be zero. The same constraint applies for coordinates where *all* entries are one. We refer to such positions as *critical* since they alone convey information that can identify the pirate. To ensure security, a fingerprinting code must support a tracing algorithm Trace that, given a pirate word $\bar{w}$ as input (with a tracing key tk), outputs the identity of at least one contributing player. That individual can then be revoked. The code is said to be c -collusion resistant if it can reliably trace pirate words formed by any coalition of up to c colluding players.

Example 1.1. Consider a fingerprinting code in Fig. 1 for $n = 4$ players and $L = 6$, 2-collusion resistant:

Player 1:	1	1	0	0	1	0
Player 2:	0	1	1	0	0	1
Player 3:	1	0	1	1	0	0
Player 4:	0	0	0	1	1	1

Figure 1: Code in Theorem 1.1

Say players 1 and 3 coalesce in Fig. 1. In a forged word $\bar{w}$, positions 1 and 6 are *critical* ones and must match values 1 and 0

respectively. Each of the other positions of $\bar{w} \in \{1 ? ? ? ? 0\}$ (denoted by “?”) can be either 0 or 1. The pattern $\{1 _ _ _ 0\}$ excludes players 2 and 4 as potential contributors. Positions 1 and 6 thus become coalition indicators of players 1 and 3 in this example.

A well-designed fingerprinting code ensures that any feasible pirate word uniquely identifies at least one member of the coalition. In our traitor tracing scheme, we will designate each user to a codeword of length L from an $n \times L$ binary matrix. When a coalition of c users builds a decoder box, it corresponds to a $c \times L$ submatrix of the original code matrix. Our tracing algorithm has a specific objective: Identify critical positions within this $c \times L$ submatrix. The algorithm achieves this through oracle access to the decoder box. Once critical positions are identified, we can trace at least one coalition member. To achieve such tracing ability in threshold encryption, Boneh et al. [13] recently formalized a specialized cryptographic tool called *Bipartite Threshold Encryption* (Bi-TE) that we discuss below. Here encryption security prevents adversary misreporting of critical positions during tracing.

Bipartite Threshold Encryption (Bi-TE) [13]. In a standard threshold encryption (TE) scheme, parameterized by (t, n) , a single encryption key is used to encrypt messages, while the corresponding decryption key is secret-shared among n participants. Each participant receives a partial decryption key, and any subset of at least t participants can produce partial decryptions of a ciphertext and jointly recover the original message.

In contrast, bipartite threshold encryption (Bi-TE) uses L encryption keys, each associated with one of L positions. To encrypt a message, the encryption algorithm selects a position $j \in [L]$ uniformly at random and uses the key at that position. At each position j , two types of decryption keys are defined: *left* and *right*. Both key types are independently secret-shared using a (t, n) scheme. Each user $i \in [n]$, however, receives only one share at position j : either a left-share or a right-share. Fig. 2 illustrates the resulting collection of $2 \times n \times L$ secret-key shares produced during key generation. Here,

	$j = 1$	$\dots$	$j = L$
$i = 1$	$(sk_{1,0}^{(1)}, sk_{1,1}^{(1)})$	$\dots$	$(sk_{1,0}^{(L)}, sk_{1,1}^{(L)})$
$\vdots$	$\vdots$	$\ddots$	$\vdots$
$i = n$	$(sk_{n,0}^{(1)}, sk_{n,1}^{(1)})$	$\dots$	$(sk_{n,0}^{(L)}, sk_{n,1}^{(L)})$

Figure 2: Secret keys in Bi-TE

Bi-TE uses fingerprinting codes to determine which secret-key type (left or right) each user receives at each position. Each user i is assigned a codeword $\bar{w}^{(i)} = \{w_1^{(i)}, \dots, w_L^{(i)}\}$, and accordingly receives exactly one key per position:

$$\left(sk_{i,w_1^{(i)}}^{(1)}, \dots, sk_{i,w_L^{(i)}}^{(L)} \right)$$

Each ciphertext similarly consists of a left and a right component. A message msg is encrypted by selecting a random index $j \leftarrow [L]$ and producing a ciphertext $ct = (j, ct_0, ct_1)$. Decryption by any quorum of t users depends on the key types assigned at position j : users holding left keys provide decryption shares for ct_0 , while those holding right keys provide shares for ct_1 . Correctness requires that *any* quorum of t keys can jointly decrypt ct , regardless

of the distribution of left and right shares among the users. We refer to this guarantee as *two-sided correctness*. For tracing, we require a complementary property, *one-sided security*. This states that an adversary holding only left keys at a given position (of any cardinality) cannot distinguish the honestly generated pair (ct_0, ct_1) from (ct_0, ct'_1) , where ct'_1 encrypts a random message msg' . A symmetric requirement holds for adversaries possessing only right keys. The Trace algorithm proceeds by replacing one component of the ciphertext with an invalid value and observing the decoder's decryption behavior across all positions. This allows the tracer to identify which codeword positions are critical for the coalition's decryption capability. Conceptually, tracing can be viewed as a semantic-security experiment: the tracer TR^D acts as the challenger, and the decoder D plays the role of a distinguisher attempting to differentiate encryptions of msg_0 and msg_1 . The tracing algorithm (using subroutine TR^D) is outlined below.

- (1) Algorithm TR^D takes three indicator bits $b_{msg}, b_{left}, b_{right}$:
 - (a) b_{msg} specifies which message (msg_0 or msg_1) the decoder is challenged on.
 - (b) If $b_{left} = 0$, the left ciphertext is an encryption of msg_0 ; otherwise it is an encryption of msg_1 .
 - (c) Similarly, if $b_{right} = 0$, the right ciphertext encrypts msg_0 ; otherwise it encrypts msg_1 .
- (2) Algorithm TR^D also takes N : the number of repetitions required for statistical majority, and Z : a threshold value for majority.
- (3) For $j = 1, \dots, L$, compute:
 - (a) $p_{001} \leftarrow TR^D(pk, j, N, 0, 0, 1) : /$ runs D with a valid left ciphertext and an invalid right ciphertext
 - (b) $p_{100} \leftarrow TR^D(pk, j, N, 1, 0, 0) : /$ runs D with invalid left and right ciphertexts
 - (c) $p_{111} \leftarrow TR^D(pk, j, N, 1, 1, 1) : /$ runs D with valid left and right ciphertexts
- (4) Set $a_0 \leftarrow |p_{001} - p_{100}|$ and $a_1 \leftarrow |p_{001} - p_{111}|$.
- (5) If $a_0 \geq Z$, set $w_j \leftarrow 0$. If not, check if $a_1 \geq Z$, and if so set $w_j \leftarrow 1$. If both conditions do not hold, set $w_j \leftarrow '?'$.
- (6) Run $J \leftarrow FC.Trace(tk, \bar{w}^*)$, where $\bar{w}^* \leftarrow w_1 w_2 \dots w_L$.

It is easy to verify that if coalition members has only left keys at position j , then a_0 will be large, allowing us to set $w_j = 0$. Similarly, if they have only right keys, then a_1 will be large, allowing us to set $w_j = 1$. To summarize, tracing operates by observing how decryption behavior varies under *different* key configurations, where the index j determines the configuration (derived from the fingerprinting code) used for encryption and decryption. Ultimately, the key insight is that effective tracing through fingerprinting requires distinct decryption responses across these varying key configurations. Next, we turn to the seemingly orthogonal challenge of batching ciphertexts for decryption, where a unified decryption response (i.e., a single decryption key) is required across multiple encryption instances that employ distinct key configurations. Achieving both batching and tracing simultaneously is non-trivial and demands a carefully crafted design, which we discuss in the following section.

Batched Threshold Encryption (BTE). We first describe non-threshold batched encryption before extending it to the threshold

setting. Consider a set of B ciphertexts $S = \{ct_1, \dots, ct_B\}$ encrypted under a public key pk . The goal of *batched encryption* (also called *batch decryption*) is to derive a succinct decryption key sk_S that decrypts all ciphertexts in S , while preserving semantic security for ciphertexts outside S . A naive approach setting $sk_S = \{sk_j \mid j \in S\}$ yields a key whose size grows linearly in $|S|$, and is thus not succinct.

We focus on batched encryption schemes in the *identity-based setting*, as recent work [2, 26] shows that batched IBE directly implies a batched public-key encryption scheme by assigning *random* identities during encryption. In a standard IBE, ciphertexts ct_{ID} (encrypting message msg) and secret keys $sk_{ID'}$ are indexed by identities $ID, ID' \in \mathcal{ID}$, and decryption succeeds iff $ID = ID'$. Identity-specific secret keys sk_{ID} are derived by an authority holding the master secret key msk . Before understanding the concrete role of identities in our batched setting, let us first briefly recall the Boneh–Franklin IBE (BF-IBE) scheme [11].

In the setup phase, the authority samples a master secret key $msk = \alpha$ and publishes the master public key $pk = g^\alpha$. To encrypt a message msg under identity ID , the sender computes $h = H(ID)$ and, using randomness r , outputs

$$\text{Enc}(pk, h, msg; r) = (ct_1, ct_2) = (g^r, e(g, h)^{r\alpha} \oplus msg).$$

To decrypt, the authority derives the identity-specific secret key $sk_{ID} = h^\alpha$, and recovers the message by computing

$$msg = ct_2 \oplus e(ct_1, h^\alpha).$$

Correctness follows from the bilinear pairing identity:

$$e(g^r, h^\alpha) = e(g, h)^{r\alpha}. \quad (1)$$

We use a color convention throughout: **blue** denotes public elements, while **red** denotes secret elements.²

In our setting, an identity-based batched encryption scheme assigns *random* identities during encryption; it also modifies the decryption condition: Instead of checking identity equality $ID = ID'$, it checks set membership if $ID' \in S$. Given a public batch of identities $S \subseteq \mathcal{ID}$ (which may be fixed after ciphertexts are created), the authority can compute a succinct secret key sk_S that decrypts all ciphertexts ct_{ID_j} for $ID_j \in S$, while ensuring that ciphertexts under identities $ID_k \notin S$ remain semantically secure. The works in [2, 26] independently realize BTE schemes by combining two main ingredients: KZG commitments [33] for set-membership proofs and Boneh–Franklin IBE for message encryption. We now outline their approach using a constraint-system (pairing-product) formulation, and later explain how to obtain an encryption scheme from this pairing equation via the witness-based encryption framework. The identity space for BTE is defined as $\mathcal{ID} := \mathcal{I}_1 \times \mathcal{I}_2$, where $\mathcal{I}_1$ is a B -element subset of $\mathbb{Z}_p$ consisting of the B -th roots of unity, and $\mathcal{I}_2 := \mathbb{Z}_p$. The encryption algorithm samples a random identity $ID = (\hat{\omega}, \varphi) \in \mathcal{ID}$ and encrypts the message msg under ID .

To derive the batch secret key sk_S for a set $S = \{ID_1, \dots, ID_B\}$, the authority first computes a digest d for S using KZG commitments. Each identity $ID_i = (\hat{\omega}_i, \varphi_i)$ is indexed so that $\hat{\omega}_i$ is the i -th root of

unity. A unique polynomial³ f of degree B is defined such that:

$$f(\hat{\omega}_i) = \varphi_i \quad \forall i \in [B].$$

The digest is the KZG commitment $g_1^{f(\tau)} \in \mathbb{G}_1$ at a secret point τ . A membership proof for an identity $ID = (\hat{\omega}, \varphi)$ is the opening:

$$\pi_{ID} = g_1^{\frac{f(\tau) - \varphi}{\tau - \hat{\omega}}} \in \mathbb{G}_1.$$

A ciphertext associated with identity ID is decryptable using

$$w = (d, \pi_{ID}, H(d)^\alpha)$$

as the witness triple iff the following two conditions hold:

- (1) π_{ID} is a valid opening of d , i.e., $ID \in S$;
- (2) $H(d)^\alpha$ is a valid BF-IBE secret key for digest d , derived from the master secret key α .

These conditions are captured by the following pairing equations:

$$e(d \cdot g_1^{-\varphi}, g_2) = e\left(\pi_{ID}, g_2^{(\tau - \hat{\omega})}\right), \quad e(H(d), g_2^\alpha) = e(H(d)^\alpha, g_2).$$

In practice, $H(d)$ is computed as $d \cdot H(bl)$, where $H(bl)$ is a random group element sampled once per batch (see [2]).

After rearranging terms, we obtain the following pairing-product constraints in witness-based encryption form:

$$\begin{aligned} e(d, g_2) \cdot e\left(\pi_{ID}, g_2^{(\hat{\omega} - \tau)}\right) &= e(g_1^\varphi, g_2), \\ e(d, g_2^\alpha) \cdot e((d \cdot H(bl))^\alpha, g_2^{-1}) &= e(H(bl)^{-1}, g_2^\alpha). \end{aligned} \quad (2)$$

Witness-Based Encryption Framework. We now describe the (pairing-based) witness encryption (WE) framework used in recent works [20, 22, 29, 30, 35]. Such a framework allows to encrypt w.r.t. a statement that a secret witness satisfies a set of pairing-product equations. In particular, say a set of such equations are given: $e(g_1^w, g_2^A) = g_2^b$, where A is a $u \times v$ matrix over a finite field and $w \in \mathbb{Z}_p^v$, $b \in \mathbb{Z}_p^u$. This formulation enables a WE scheme for the statement: “There is a witness (namely, g_1^w) satisfying $e(\cdot, g_2^A) = g_2^b$.” In such systems, a message msg can be encrypted by sampling $r \leftarrow \mathbb{Z}_p^u$ and computing the ciphertext $ct = (ct_1, ct_2)$, where $ct_1 = g_2^{r^T \cdot A}$ and $ct_2 = g_2^{r^T \cdot b} \oplus msg$. Given a valid witness g_1^w , the message can be recovered by computing $ct_2 \oplus e(g_1^w, ct_1)$.

One can verify that BF-IBE fits directly into this framework via Eq. (1). Likewise, the pairing equations in Eq. (2) describing batch encryption can be expressed in this WE framework. Recall from Eq. (2) that, among the witness elements, the digest d and the membership proof π_{ID} depend only on the batch of identities and are publicly computable, whereas the element $(d \cdot H(bl))^\alpha$ is derived from the master secret key α . Thus, the decryption key for a batch is succinctly represented by a single group element. So, to obtain a batched *threshold* encryption scheme BTE, one simply secret-shares the master secret key α across n parties using a (t, n) -threshold scheme. Since the underlying WE instance requires only the value $(d \cdot H(bl))^\alpha$, thresholdizing the evaluation of this exponentiation yields an immediate thresholdized version of the construction.

²In some derivations, red may also denote elements that are publicly reconstructible during the decryption process.

³There are two standard interpretations for constructing the polynomial: using the identities as roots, or using them as evaluation points. The latter is FFT-friendly and is used in [2, Figures 4], [26].

Overview of Bipartite Batched Threshold Encryption (Bi-BTE).

We now provide an overview of our solution. Our ultimate goal is to integrate tracing into batched threshold encryption systems. To achieve this, we introduce *bipartite batched threshold encryption* (Bi-BTE), which carefully combines the essential properties of batched threshold encryption (BTE) and bipartite threshold encryption (Bi-TE). Challenge is that batching and tracing appear to have conflicting requirements. Batching requires a unified decryption for all ciphertext in a batch which is achieved through an encryption by a single master public key but varying different identities per ciphertext. While decryption is performed using a single batch secret key derived from the master secret key. Here single master public/secret key pair is essential for uniform decryption behavior across ciphertexts in a batch. Tracing, on the other hand, requires varying decryption behavior across ciphertexts, which is achieved through distinct key configurations per ciphertext. Recall our discussion on tracing in Bi-TE above, the trace algorithm by changing the key configurations (determined by index j) and analyzing the resulting decryption behavior. If particular position j has only left keys in the coalition, then the decryption behavior will be different from the case when it has only right keys. In each position, different left and right keys are assigned to different users based on the fingerprinting code. Thus, tracing relies on observing decryption behavior where varying key configurations (determined by index j) are essential for effective tracing. Not all hope is lost though. We overcome this by observing the following: tracing can still be performed across *different* batches by *varying* the master keys *between* batches, while *fixing* the master keys *per batch*. We need following three steps to achieve this:

- (1) An identity based bipartite encryption scheme to support two-sided correctness and one-sided security.
- (2) Converting the bipartite encryption scheme to the witness-based encryption framework to support batch decryption.
- (3) A position sampling mechanism to ensure that each batch uses a specific index j for correctness.

We build upon the bipartite threshold identity-based encryption scheme of Bhattacharya et al. [8] for the first step. We adopt the high-level idea from [8] how to convert BF-IBE (see Eq. (1)) to bipartite setting. The setup algorithm sample $\alpha, \gamma, z \in \mathbb{Z}_p$ and set the master public key as $pk = (X = g_2^{\alpha\gamma z}, Y = g_2^\gamma, Z = g_2^z)$. Master secret key is divided into left-side and right-side secret keys as $sk_0 = z\alpha$ and $sk_1 = \gamma\alpha$ respectively. For an identity ID, the encryption algorithm samples $r \in \mathbb{Z}_p$ and computes the ciphertext as $ct = \{e(h, X)^r \oplus msg, Y^r, Z^r\}$ where $h = H(ID)$. Here ciphertext are divided into left and right parts as $ct_L = Y^r$ and $ct_R = Z^r$. The decryption algorithm uses either left-side or right-side secret key to decrypt the corresponding part of the ciphertext. It is easy to verify that either left-side or right-side secret key can be used to compute the pairing expression below for decryption:

$$e(h^{z\alpha}, Y^r) = e(h, g_2)^r \text{ or } e(h^{\gamma\alpha}, Z^r) = e(h, X)^r \quad (3)$$

One sided security arises because the left-side secret key $sk_0 = z\alpha$ does not aid in distinguishing the right part of the ciphertext Z^r from a random ciphertext Z' . It is straightforward to thresholdize the above scheme by secret-sharing α into $\{s_i\}_{i \in [n]}$ using an (t, n) -Shamir's-secret-sharing scheme. The left-side and right-side secret

keys for party i are given by $sk_{i,0} = zs_i$ and $sk_{i,1} = \gamma s_i$ respectively. Encryption is similarly defined as above. The decryption uses any combination of left and right secret keys to compute the pairing expression below:

$$\prod_{i \in \mathbb{L}} e(h^{zs_i \ell_i}, Y^r) \cdot \prod_{i \in \mathbb{R}} e(h^{\gamma s_i \ell_i}, Z^r) = e(h, X)^r \quad (4)$$

Here, $\mathbb{L}$ and $\mathbb{R}$ denote the left and right coalition sets respectively, and ℓ_i are Lagrange coefficients. In a full-fledged bipartite threshold encryption scheme, $[L]$ such independent instances are generated by sampling independent master keys $\{(\alpha_j, \gamma_j, z_j)\}_{j \in [L]}$. The encryption and decryption algorithms operate similarly as above for a fixed position j . We extend the above bipartite threshold IBE to support batch decryption using the witness-based encryption framework. The key observation is that the pairing expression in Eq. (4) can be adapted to support batch decryption by replacing the identity-specific hash h with a batch-specific digest d combined with a batch-specific randomizer $H(j, bl)$ (where j is a position index and bl is the batch label). The digest d and membership proof are computed similarly to how they are generated in BTE (as discussed above). Similar to Eq. (2) for batched encryption, we modify the pairing expression in Eq. (4) as follows to support batch decryption and thresholdization:

$$\begin{aligned} e(d, g_2) \cdot e(\pi_D, g_2^{(\hat{\omega}-\tau)}) &= e(g_1^{\phi}, g_2) \\ e(d, X_j) \cdot e(\sigma^{(0)}, Y_j^{-1}) \cdot e(\sigma^{(1)}, Z_j^{-1}) &= e(H(bl)^{-1}, X_j) \end{aligned} \quad (5)$$

where $\sigma^{(0)} = \prod_{i \in \mathbb{L}} (d \cdot H(j, bl))^{(s_{ij} \ell_i) z_j}$ and $\sigma^{(1)} = \prod_{i \in \mathbb{R}} (d \cdot H(j, bl))^{(s_{ij} \ell_i) \gamma_j}$ denote the left and right combined decryption keys, respectively. It is easy to see that the above equations can be expressed in the witness-based encryption framework, i.e., $e(g_1^w, g_2^A) = g_2^b$, leading to an encryption scheme. As our original batched decryption scheme requires a random batch-specific label bl , similarly bipartite batched decryption also requires a random index j . The final step is to ensure that each batch of ciphertexts uses a specific index j for decryption correctness. To maintain consistency in decryption across all ciphertexts within a batch, we employ a position sampling mechanism to derive the index j from bl . This index is generated by a position sampling algorithm $\text{PosSamp}(bl)$, which we realize using a hash function $H_{idx} : \{0, 1\}^* \rightarrow [L]$, such that $j = H_{idx}(bl)$. We adapt a variant of the IBE-to-CCA generic transformation [10, 21] using one-time signatures to construct a CCA-secure Bi-BTE scheme. Alike [8], we separate out the tag key generation and tag creation from encryption: $(sk_{OTS}, vk_{OTS}) \leftarrow \text{TagKeys}(1^\lambda, pp)$ and $tg \leftarrow \text{GenTag}(sk_{OTS}, ct)$. This separation of tag generation and creation is necessary as in one-sided security (see Fig. 6) we replace part of the ciphertext (either left or right) with random ciphertext. To achieve this, the challenger generates a single key pair of (sk_{OTS}, vk_{OTS}) and produces two ciphertexts via independent encryption operations under the same identity, i.e., the verification key vk_{OTS} . The tracer then uses GenTag to generate a valid tag for the malformed ciphertext, leveraging the fact that both original ciphertexts were created under the same identity vk_{OTS} .

Formalizing Traceable Batched Threshold Encryption (T-BTE).

To move towards our final goal, we define the notion of traceable

batched threshold encryption (T-BTE) by *carefully melding* batching functionality with tracing capabilities. Our T-BTE scheme is constructed generically from Bi-BTE and a fingerprinting code. The compiler follows a similar approach to the one proposed by Boneh et al. [13] for lifting Bi-TE to Traceable Threshold Encryption (T-TE). The key difference in our tracing algorithm is that the decoder now receives a batch of ciphertexts as input, necessitating a revised definition of the decoder (see Definition 3.5). The T-BTE key generation algorithm samples a fingerprinting code consisting of n codewords, each of length L , and then invokes the Bi-BTE key generation algorithm with input L . All T-BTE algorithms primarily rely on the underlying Bi-BTE subroutines, except for the tracing algorithm. The tracing algorithm must traverse all indices $j \in [L]$ while simultaneously feeding a batch of ciphertexts to the decoder. Unlike encryption algorithm, our tracing algorithm does not derive the position from the batch label ($j = \text{PosSamp}(\text{bl})$). Instead, the tracing algorithm explicitly chooses an index j , then samples a random batch label bl , ensuring consistency by fixing j for each batch. We outlined the intuition behind the tracing mechanism in the paragraph on “Bipartite Threshold Encryption” in Sec. 1.3. The tracer constructs combinations of valid and malformed batch of ciphertexts by mixing left and right components from two distinct ciphertexts. Based on whether decryption succeeds or fails, the tracer recovers the pirate codewords and invokes the tracing algorithm of the underlying fingerprinting code.

1.4 Related Work

We succinctly summarize the works relevant to (threshold) traitor-tracing and batched threshold encryption. We summarize and contrast features of all these recent works in Table 1.

Traitor-Tracing. Traitor-tracing (TT) has been studied widely spanning across the last three decades [14, 15, 24, 32]. The notion of *threshold* traitor-tracing (TTT) we study in this paper was recently initiated and studied by Boneh et al. [13]. They built several TTT schemes, all in the centralized setting (with a trusted authority) supporting secret tracing. Recently, Bormet et al. [19] defined and built traceable silent threshold encryption (TSTE) again with secret tracing. Here, silent means absence of any trusted authority. Concurrent to [19], Chakrabarti et al. [23] formalized the notion of silent TTT (with public tracing) providing two realizations from bilinear pairings. But none of these systems support batching ciphertexts. Further, the schemes in [23] does not satisfy CCA security.

Batched Threshold Encryption and Accountability. Due to its immediate boost on the efficiency in blockchain systems, the notion of BTE [2, 17, 25, 26] has recently garnered significant interest as a promising approach to achieve mempool privacy [6]. Recently, Bormet et al. [16] built a BTE scheme with silent setup [31]. Some other recent works have also studied and provided BTE constructions in various settings [2, 17, 25, 26]. Table 1 summarizes and contrasts features of all these recent works. However, one persistent major issue in all these works is the lack of *accountability* in such systems. We attempt to address this issue in this work for the *first* time by formalizing the notion of T-BTE and providing a concrete construction using fingerprinting codes [12] and BTE from [2]. Our T-BTE scheme is CCA-secure. But it supports secret

tracing; so it requires a trusted setup (or, a distributed key generation (DKG) with a trusted *tracing authority*). We leave building a silent BTE with public tracing as an interesting future work.

2 Preliminaries

In this section, we introduce the notation, building blocks, and models used in this paper. Due to space constraints, we describe standard notations and details on bilinear pairings in Appendix A.

Building Blocks. We rely on the following cryptographic building blocks: (i) One-time signature schemes $\text{OTS} = (\text{KGen}, \text{Sign}, \text{Vf})$, (ii) Shamir Secret Sharing $\text{SSS} = (\text{Share}, \text{Recon})$, and (iii) Fingerprinting codes $\text{FC} = (\text{Gen}, \text{Trace})$. Due to space constraints, we defer the detailed description of the building blocks preliminaries in Appendix A.

3 Defining Traceable BTE with CCA-security

In this section, we formally define the notion of Traceable Batched Threshold Encryption (T-BTE).

Definition 3.1 (Traceable Batched Threshold Encryption). A traceable batched threshold encryption scheme T-BTE consists of the following PPT algorithms (KGen , Enc , Digest , ComputePartKey , CombPartKey , BDec , Trace) which have the following syntax:

- $(\{sk_i\}_{i \in [n]}, pk, tk) \leftarrow \text{KGen}(1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^{1/\epsilon})$: The randomized key generation algorithm takes as input the security parameter λ and the maximum batch size $B_{\max}$, the total number of parties n , the corruption threshold t such that $0 < t \leq n$ and an error bound $\epsilon = \epsilon(\lambda)$ which is essentially a lower bound on the distinguishing advantage of decoders that must be traceable by this scheme. It outputs a set of secret keys (partial master secret keys) $\{sk_i\}_{i \in [n]}$, a single public key pk (global public key) and a tracing key tk . Each party uses its partial master secret key to generate identity-based keys (for partial decryption), while the public key is used for encryption.
- $(ct, id) \leftarrow \text{Enc}(pk, msg, bl)$: The randomized encryption algorithm takes as input a public key pk , a message $msg \in \mathcal{M}$ and a batch label $bl \in \mathcal{B}$. It outputs a ciphertext $ct \in \mathcal{C}$, an identity $id \in \mathcal{ID}$ and a tag tg .
Note: To get CCA security, we deviate from the standard definition of IBE, where the encryption algorithm typically takes an identity id as input. Instead, our encryption algorithm samples a verification key and a signing key $(sk_{\text{OTS}}, vk_{\text{OTS}})$ from one-time signature, and then encrypts the message msg with respect to the “identity” vk_{OTS} . Thus, the identity id is part of the output, rather than being provided as input.
- $d \leftarrow \text{Digest}(pk, \{id_i\}_{i \in [B]})$: The deterministic digest algorithm that takes as input the global public key pk and a set of identities $\{id_1, \dots, id_B\}$ where each $id_i \in \mathcal{ID}$, and outputs a digest d of the set of identities.
- $d_i / \perp \leftarrow \text{ComputePartKey}(i, sk_i, d, bl)$: The deterministic partial batch decryption algorithm that takes as input an index $i \in [n]$, and its corresponding secret key sk_i , a digest d and a batch label bl . It outputs the batch label specific partial decryption key d_i , otherwise $\perp$. The algorithm ComputePartKey serves as a secret key generation procedure, producing both partial(in a

Scheme	Setup	Batched (Yes/No)	Tracing (Secret/Public)	CCA Security	crs	ek or mpk	sk	ct
TE [31]	Silent	No	×	✓	$O(n)$	$O(1)$	$O(1)$	$O(1)$
BTE [26]	Centralized	Yes	×	✓	–	$O(B)$	$O(1)$	$O(1)$
BTE [17]	Centralized	Yes	×	✓	–	$O(B)$	$O(1)$	$O(1)$
BTE [2]	Centralized	ST	×	✓	–	$O(B)$	$O(1)$	$O(1)$
BTE [16]	Silent	Yes	×	✓	$O(n+B)$	$O(1)$	$O(1)$	$O(\lambda)$
T-TE [13]	Centralized	No	✓ (Secret)	×	–	$O(n^2 \log^2 n)$	$O(n^2 \log^2 n)$	$O(1)$
T-TE [8]	Centralized	No	✓ (Secret)	✓	–	$O(n^2 \log^2 n)$	$O(n^2 \log^2 n)$	$O(1)$
T-TE [19]	Silent	No	✓ (Secret)	×	$O(n)$	$O(n^4)$	$O(1)$	$O(1)$
T-TE [23]	Silent + Transparent	No	✓ (Public)	×	$O(1)$	$O(\sqrt{n})$	$O(1)$	$O(\sqrt{n})$
Our T-BTE	Centralized	Yes	✓ (Secret)	✓	–	$O(n^2 \log^2 n)$	$O(n^2 \log^2 n)$	$O(1)$

Table 1: Comparison of features from prior work and our schemes. TE, BTE, and T-TE denote threshold encryption, batched threshold encryption, and traceable threshold encryption, respectively. A transparent setup corresponds to having an unstructured crs. Here, n denotes the number of users, B denotes the batch size, and λ denotes the security parameter.

threshold scheme) and identity-based secret keys for a batch of identities(digest).

- $\sigma/\perp \leftarrow \text{CombPartKey}(\mathcal{J}, \text{bl}, \{d_i\}_{i \in \mathcal{J}}, d)$: The deterministic batch decryption combine algorithm that takes as input combine set $\mathcal{J} \geq t$, batch label bl, batch label specific set of partial decryption keys $\{d_i\}_{i \in \mathcal{J}}$ and a digest d. It outputs σ , else $\perp$.
- $\{\text{msg}_i\}_{i \in [B]} \leftarrow \text{BDec}(\{(ct_i, id_i)\}_{i \in [B]}, \text{bl}, \sigma, d)$: The deterministic batch decryption algorithm that takes as input a batch of ciphertext-identity pairs $\{(ct_i, id_i)\}_{i \in [B]}$, a batch label bl, value σ , a digest d. It outputs the messages $\{\text{msg}_i\}_{i \in [B]} \in \mathcal{M}$.
- $\mathcal{J}' \leftarrow \text{Trace}^D(\text{pk}, \text{tk}, 1^{1/\epsilon(\lambda)})$: The trace algorithm having oracle access to a “decoder” D, inputs a public key pk a trace key tk and an error bound $\epsilon = \epsilon(\lambda)$. It outputs a set $\mathcal{J}' \subseteq [n]$.

A traceable batched threshold encryption scheme T-BTE must satisfy the properties of *correctness* (Definition 3.2), *efficiency* (Definition 3.3), *IND-CCA security* (Definition 3.4) and *tracing security* (Definition 3.5) described below.

Definition 3.2 (Correctness). A traceable batched threshold encryption scheme T-BTE is correct if for any security parameter $\lambda \in \mathbb{N}$, batch size $B \in \mathbb{N}$, the total number of parties $n \in \mathbb{N}$, threshold $t \in \mathbb{N}$ such that $0 < t \leq n$ and $\text{msg} \in \mathcal{M}$, $\{id_i\}_{i \in [B]} \in \mathcal{ID}$, coalition set $\mathcal{J} \subseteq [n]$ such that $|\mathcal{J}| \geq t$, batch label $\text{bl} \in \mathcal{B}$, the following probability is 1.

$$\Pr \left[\begin{array}{l} \forall i \in [B] \\ \text{msg}'_i = \text{msg}_i \end{array} \middle| \begin{array}{l} (\text{pk}, \{sk_i\}_{i \in [n]}, \text{tk}) \leftarrow \text{KGen}(1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^{1/\epsilon}) \\ (ct_i, id_i) \leftarrow \text{Enc}(\text{pk}, \text{msg}_i, \text{bl}) \forall i \in [B] \\ d \leftarrow \text{Digest}(\text{pk}, \{id_i\}_{i \in [B]}) \\ d_i \leftarrow \text{ComputePartKey}(i, sk_i, d, \text{bl}) \forall i \in \mathcal{J} \\ \sigma \leftarrow \text{CombPartKey}(\mathcal{J}, \text{bl}, \{d_i\}_{i \in \mathcal{J}}, d) \\ \{\text{msg}'_i\}_{i \in [B]} \leftarrow \text{BDec}(\{(ct_i, id_i)\}_{i \in [B]}, \text{bl}, \sigma, d) \end{array} \right]$$

Definition 3.3 (T-BTE Efficiency). For non-trivial efficiency, we require that the running time of ComputePartKey and CombPartKey be independent of the batch size B. This implies that both the digest d and each partial decryption key d_i must be independent of B. Additionally, we require that the running time of ComputePartKey and the size of the combined key σ be independent of the number of authorities n .

Definition 3.4 (IND-CCA Security). A traceable batched threshold encryption scheme T-BTE is IND-CCA secure if for all $\lambda \in \mathbb{N}$, $B \in \mathbb{N}$, $n \in \mathbb{N}$, $t \in \mathbb{N}$ such that $0 < t \leq n$, and for all $\text{Corr} \subset [n]$ s.t. $|\text{Corr}| < t$ and if the advantage of any PPT adversaries $\mathcal{A}$ defined by $\text{Adv}_{\text{T-BTE}, \mathcal{A}}^{\text{IND-CCA}}$ in $\text{IND-CCA}_{\mathcal{A}}^{\text{T-BTE}}$ (Fig. 3) is negligible.

$$\text{Adv}_{\text{T-BTE}, \mathcal{A}}^{\text{IND-CCA}} = \Pr[\text{IND-CCA}_{\mathcal{A}}^{\text{T-BTE}}(1^\lambda, B, n, t, \text{Corr}) = 1] \leq \frac{1}{2} + \text{negl}$$

IND-CCA $_{\mathcal{A}}^{\text{T-BTE}}(1^\lambda, B, n, t, \text{Corr})$

```

1: Initialize  $Q_1, Q_2, Q_3 \leftarrow \emptyset$ 
2:  $(\{sk_i\}_{i \in [n]}, \text{pk}, \text{tk}) \leftarrow \text{KGen}(1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^{1/\epsilon})$ 
3:  $(\text{msg}_0, \text{msg}_1, \text{bl}^*, \text{st}) \leftarrow \mathcal{A}^{\text{KeyDer}}(\text{pp}, \text{pk}, \text{tk}, \{sk_i\}_{i \in \text{Corr}})$ 
4: Sample  $b \leftarrow \{0, 1\}$ 
5:  $(ct^*, id^*) \leftarrow \text{Enc}(\text{pk}, \text{msg}_b, \text{bl}^*)$ 
6:  $b' \leftarrow \mathcal{A}^{\text{BDec, KeyDer}}(\text{st}, ct^*, id^*)$ 
   / No key-computation query on the challenge batch label
   / where challenge identity is present as part of input
7: if  $\text{bl}^* \in Q_1 \wedge id^* \in Q_2(\text{bl}^*)$ : return 0
   / No Decryption query on the challenge(ciphertext, identity, batch label)
   /  $\mathcal{A}$  can potentially query on the challenge (batch label, identity) but different ciphertext
8: if  $(ct^*, id^*) \in Q_3$ : return 0
9: return 1 if  $b = b'$ , else return 0

KeyDer( $\{id_i\}_{i \in [B]}, \text{bl}$ )
1: if  $\{\text{bl}\} \in Q_1$  then return  $\perp$ 
2:  $d \leftarrow \text{Digest}(\text{pk}, \{id_i\}_{i \in [B]})$ 
3:  $d_i \leftarrow \text{ComputePartKey}(i, sk_i, d, \text{bl}) \forall i \in [n] \setminus \text{Corr}$ 
4:  $Q_1 \leftarrow Q_1 \cup \{\text{bl}\}$ 
5:  $Q_2(\text{bl}) \leftarrow Q_2(\text{bl}) \cup \{\{id_i\}_{i \in [B]}\}$ 
6: return  $\{d_i\}_{i \in [n] \setminus \text{Corr}}$ 

BDec( $\{(ct_i, id_i)\}_{i \in [B]}\rangle$  / only for  $\text{bl}^*$ )
1:  $d \leftarrow \text{Digest}(\text{pk}, \{id_i\}_{i \in [B]})$ 
2:  $d_i \leftarrow \text{ComputePartKey}(i, sk_i, d, \text{bl}^*) \forall i \in [n]$ 
3:  $\sigma \leftarrow \text{CombPartKey}([n], \text{bl}^*, \{d_i\}_{i \in [n]}, d)$ 
4:  $\{\text{msg}_i\}_{i \in [B]} \leftarrow \text{BDec}(\{(ct_i, id_i)\}_{i \in [B]}, \text{bl}^*, \sigma, d)$ 
5:  $Q_3 \leftarrow Q_3 \cup \{\{(ct_i, id_i)\}_{i \in [B]}\}$ 
6: return  $\{\text{msg}_i\}_{i \in [B]}$ 

```

Figure 3: IND-CCA security game for T-BTE scheme.

Tracing Security. We adapt the formal definition of tracing security from [13]. Informally, if the decoder D, given a batch of ciphertexts $\{ct_i^b\}$, can learn any information about the underlying batch of messages (captured in GoodDec), then Trace traces D back to a party in the coalition that “manufactured” it. The following definition formalizes this property using the security experiment in Figure 4. The security game allows the adversary to corrupt a set of parties $\mathcal{J}$ of its choosing. The event GoodTr occurs when Trace successfully traces back one of the parties in $\mathcal{J}$, and BadTr occurs when tracing wrongfully accuses someone the adversary did not

corrupt. Note that in our definition of tracing security, the tracing key is kept secret from the adversary. This is since, for our schemes, knowledge of this key would allow the adversary to evade tracing.

Definition 3.5 (Tracing Security). A traceable batched threshold encryption scheme T-BTE is tracing secure if for all PPT adversaries $\mathcal{A}$, the following conditions hold: $\Pr[\text{GoodTr}] \geq \Pr[\text{GoodDec}] - \delta(\lambda)$ and $\Pr[\text{BadTr}] \leq \delta(\lambda)$ where $(D, \mathcal{J}, \mathcal{J}') \leftarrow \text{TrSec}_{\mathcal{A}}^{\text{T-BTE}}(1^\lambda)$ as defined in Fig. 4. The events GoodDec, GoodTr, and BadTr are as follows:

- GoodDec occurs when $P(D) \geq \epsilon(\lambda)$, where D is the decoder. We define $P(D)$, distinguishing probability between two B -sized randomly picked message vectors from the message space as follows:

$$P(D) = \Pr \left[D(\{ct_i^b, id_i\}_{i \in [B]}) = b \mid \begin{array}{l} b \leftarrow \{0, 1\}; bl \leftarrow \mathcal{B}; B \leftarrow [B_{\max}] \\ (ct_i^0, id_i) \leftarrow \text{Enc}(pk, msg_i^0, bl) \forall i \in [B] \\ (ct_i^1, id_i) \leftarrow \text{Enc}(pk, msg_i^1, bl) \forall i \in [B] \end{array} \right]$$

- GoodTr holds when $\mathcal{J} \neq \emptyset$ and $\mathcal{J}' \subseteq \mathcal{J}$.
- BadTr holds when $\mathcal{J} \neq \emptyset$ and $\mathcal{J}' \not\subseteq \mathcal{J}$.

We say that T-BTE is tracing secure if there exists a negligible function $\nu = \nu(\lambda)$ such that T-BTE is $(1/p, \nu)$ -tracing secure for every polynomial $p = p(\lambda)$. For an adversary $\mathcal{A}$ and an T-BTE scheme, we define the advantage:

$$\text{Adv}_{\text{T-BTE}, \mathcal{A}}^{\text{TrSec}} = \max \{ \Pr[\text{GoodDec} = 1] - \Pr[\text{GoodTr} = 1], \Pr[\text{BadTr}] \}$$

$\text{TrSec}_{\mathcal{A}}^{\text{T-BTE}}(1^\lambda)$	
1:	$(n, t, st_1) \leftarrow \mathcal{A}(1^\lambda)$
2:	$(\{sk_i\}_{i \in [n]}, pk, tk) \leftarrow \text{KGen}(1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^{1/\epsilon})$
3:	$(\mathcal{J}, st_2) \leftarrow \mathcal{A}(st_1, pp, pk)$
4:	$D \leftarrow \mathcal{A}(st_2, \{sk_i\}_{i \in \mathcal{J}})$
5:	$\mathcal{J}' \leftarrow \text{Trace}^D(pk, tk, 1^{1/\epsilon(\lambda)})$
6:	return $(D, \mathcal{J}, \mathcal{J}')$

Figure 4: TrSec security game for T-BTE scheme.

4 Bipartite BTE with CCA-security

In this section, we provide a detailed syntax and security definition of our proposed primitive Bipartite Batched Threshold Encryption (Bi-BTE), along with a construction and detailed security analysis of our Bi-BTE construction.

A Bi-BTE is an encryption scheme in which, instead of a single set of secret keys, there are $2L$ set of secret keys, determined by a scheme parameter L . These keys are arranged in a $n \times L$ matrix, where each row corresponds to a party and each column denotes a position. For each position, there exist two possible secret keys per party, denoted by a left key and a right key. Similarly, a ciphertext can be logically divided into two components, denoted by a left ciphertext and a right ciphertext. In a batched threshold encryption scheme, the encryption algorithm associates a unique identifier with each batch. Prior work refers to this identifier either as a batch label [2] or as an epoch [26]. Furthermore, in the bipartite threshold encryption scheme [13], the encryption algorithm is required, for security purposes, to sample a uniformly random index value. Since it is necessary for to fix the random index value for a given batch, we unify these requirements by introducing a deterministic algorithm, denoted PosSamp. This algorithm takes a random batch label as

input and outputs a corresponding position for that batch. The procedure can be instantiated using a cryptographic hash function. Bipartite encryption admits a special class of malformed ciphertexts in which one component (e.g., the left ciphertext) is valid with respect to a given message, while the other component (e.g., the right ciphertext) is invalid, having originated from the encryption of a different message. Our technique for handling such malformed ciphertexts is adapted from [8]. To establish that CCA security remains preserved in this setting, it is necessary to ensure the existence of a valid tag for such malformed ciphertexts, even though they arise from two distinct encryption executions. Consequently, in our formulation, the tag key generation algorithm is decoupled from the core encryption algorithm. This separation enables the generation of valid tags for malformed ciphertexts and thereby supports the security argument.

4.1 Syntax and Security Definition of Bi-BTE

Definition 4.1 (Bipartite Batched Threshold Encryption). A bipartite batched threshold encryption scheme Bi-BTE consists of the following tuple of PPT algorithms (Setup, KGen, TagKeys, PosSamp, Enc, Digest, ComputePartKey, CombPartKey, BDec) which have the following syntax:

- $pp \leftarrow \text{Setup}(1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^L)$: The setup algorithm takes as input the security parameter λ and the maximum batch size $B_{\max}$, the total number of parties n , the corruption threshold t such that $0 < t \leq n$ and the number of positions $L \in \mathbb{N}$. It outputs public parameters pp which includes a description of the message space $\mathcal{M}$, identity space $\mathcal{ID}$, batch label space $\mathcal{B}$ and ciphertext space $\mathcal{C}$. We will assume w.l.o.g. that public parameter pp is implicitly known to the rest of the algorithms.
- $(\{sk_{i,0}^{(j)}, sk_{i,1}^{(j)}\}_{j \in [L], i \in [n]}, pk) \leftarrow \text{KGen}(pp)$: The key generation algorithm takes as input the public parameters pp , and outputs a set of secret keys (partial master secret keys) $\{sk_i\}_{i \in [n]}$ and a single public key pk (global public key). Each secret key $sk_i := \{sk_{i,0}^{(j)}, sk_{i,1}^{(j)}\}_{j \in [L]}$ is made up of $2L$ keys. $sk_{i,0}^{(j)}$ is called left secret key of i at position j , and similarly $sk_{i,1}^{(j)}$ is called right secret key of i at position j . Each party uses its partial master secret key to generate identity-based keys (for partial decryption), while the public key is used for encryption.
- $(sk_{OTS}, vk_{OTS}) \leftarrow \text{TagKeys}(1^\lambda)$: The tag key generation algorithm takes as input the security parameter and outputs a key pair (sk_{OTS}, vk_{OTS}) that is used to generate tags for ciphertexts.
- $j \leftarrow \text{PosSamp}(bl)$: The position sample algorithm takes as input a batch label $bl \in \mathcal{B}$ and outputs a position $j \in [L]$.
- $(ct = (ct_L, ct_R), id) \leftarrow \text{Enc}(pk, msg, j, bl, vk_{OTS})$: The encryption algorithm takes as input a public key pk , a message $msg \in \mathcal{M}$, a position $j \in [L]$, a batch label $bl \in \mathcal{B}$ and a tag key vk_{OTS} . It outputs a ciphertext $ct \in \mathcal{C}$, an identity $id \in \mathcal{ID}$.
Note: We deviate from the standard IBE definition here. In particular, our encryption algorithm will output a *randomly sampled* identity. (Moving ahead, in our construction, we first sample tag keys and then derive the identity from them similar to the IBE-to-CCA transformation in [10, 21] also adapted in [8].)
- $tg \leftarrow \text{GenTag}(sk_{OTS}, ct)$: The tag generation algorithm inputs the secret key sk_{OTS} and a ciphertext ct , and outputs a tag tg .

- $d \leftarrow \text{Digest}(\text{pk}, \{\text{id}_i\}_{i \in [B]})$: The deterministic digest algorithm inputs the public key pk and a set of identities $\{\text{id}_1, \dots, \text{id}_B\}$ where each $\text{id}_i \in \mathcal{ID}$, and outputs a digest d of the set of identities.
- $d_i/\perp \leftarrow \text{ComputePartKey}(i, j, \text{sk}_{i,b}^{(j)}, d, \text{bl})$: The deterministic partial batch decryption algorithm that inputs an index $i \in [n]$, position j and its corresponding secret key $\text{sk}_{i,b}^{(j)}$ - where $j \in [L]$, $i \in [n]$, $b \in \{0, 1\}$, a digest d and a batch label bl . It outputs the batch bl and position j specific partial decryption d_i , otherwise $\perp$. We may assume d_i encodes the bit b even if this is not explicitly noted. The algorithm ComputePartKey serves as a secret key generation procedure, producing both partial(in a threshold scheme) and identity-based secret keys for a batch of identities.
- $\sigma/\perp \leftarrow \text{CombPartKey}(\mathcal{J}, j, \text{bl}, \{d_i\}_{i \in \mathcal{J}}, d)$: The deterministic batch decryption combine algorithm inputs a set $\mathcal{J} \geq t$, position j , batch label bl , batch bl and position j specific set of partial decryption $\{d_i\}_{i \in \mathcal{J}}$ and a digest d . It outputs σ , otherwise $\perp$.
- $\{\text{msg}_i\}_{i \in [B]} \leftarrow \text{BDec}(\{\text{ct}_i, \text{id}_i, \text{tg}_i\}_{i \in [B]}, j, \text{bl}, \sigma, d)$: The deterministic batch decryption algorithm that takes as input a batch of ciphertext-identity pairs $\{(\text{ct}_i, \text{id}_i)\}_{i \in [B]}$, a position j , a batch label bl , value σ , a digest d . It outputs messages $\{\text{msg}_i\}_{i \in [B]} \in \mathcal{M}$.

A bipartite batched threshold encryption with CCA security scheme Bi-BTE must satisfy the properties of *two-sided correctness* (Definition 4.2), *efficiency* (Definition 4.3), *IND-CCA security* (Definition 4.4) and *one-sided security* (Definition 4.5) described below.

Definition 4.2 (Two-Sided Correctness). A bipartite batched threshold encryption scheme Bi-BTE = (Setup, KGen, TagKeys, PosSamp, Enc, GenTag, Vf, Digest, ComputePartKey, CombPartKey, Dec, BDec) is correct if for any security parameter $\lambda \in \mathbb{N}$, batch size $B \in \mathbb{N}$, the total number of parties $n \in \mathbb{N}$, threshold $t \in \mathbb{N}$ s.t. $0 < t \leq n$ and $\text{msg} \in \mathcal{M}$, $\{\text{id}_i\}_{i \in [B]}$, coalition set $\mathcal{J} \subseteq [n]$ s.t. $|\mathcal{J}| \geq t$, batch label $\text{bl} \in \mathcal{B}$ and n -bit string $s \in \{0, 1\}^n$, the following probability is 1.

$$\Pr \left[\begin{array}{l} \forall i \in [B] \\ \text{msg}'_i = \text{msg}_i \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^L) \\ ((\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})_{j \in [L], i \in [n]}, \text{pk}) \leftarrow \text{KGen}(\text{pp}); j \leftarrow \text{PosSamp}(\text{bl}) \\ \forall i \in [B] : (\text{sk}_{\text{OTS},i}, \text{vk}_{\text{OTS},i}) \leftarrow \text{TagKeys}(1^\lambda); \\ \forall i \in [B] : (\text{ct}_i, \text{id}_i) \leftarrow \text{Enc}(\text{pk}, \text{msg}_i, j, \text{bl}, \text{vk}_{\text{OTS},i}) \\ \forall i \in [B] : \text{tg}_i \leftarrow \text{GenTag}(\text{sk}_{\text{OTS},i}, \text{ct}_i) \\ d \leftarrow \text{Digest}(\text{pk}, \{\text{id}_i\}_{i \in [B]}) \\ \forall i \in \mathcal{J} : d_i \leftarrow \text{ComputePartKey}(i, j, \text{sk}_{i,j}^{(j)}, d, \text{bl}) \\ \sigma \leftarrow \text{CombPartKey}(\mathcal{J}, j, \text{bl}, \{d_i\}_{i \in \mathcal{J}}, d) \\ \{\text{msg}'_i\}_{i \in [B]} \leftarrow \text{BDec}(\{(\text{ct}_i, \text{id}_i, \text{tg}_i)\}_{i \in [B]}, j, \text{bl}, \sigma, d) \end{array} \right]$$

A coalition holding t secret keys, denoted $\text{sk}_{i,s_j}^{(j)}$, must be able to decrypt the ciphertext. This requirement must hold regardless of how many of these keys are left or right keys (i.e., for any arbitrary string s). This condition is referred to as two-sided correctness.

Definition 4.3 (Bi-BTE Efficiency). For non-trivial efficiency, we need the running time of ComputePartKey and CombPartKey to be independent of the batch size B . This implies that both the digest d and each partial decryption d_i must be independent of B . Additionally, we need the running time of ComputePartKey and the size of σ to be independent of the number of authorities n .

Definition 4.4 (IND-CCA Security). A bipartite batched threshold encryption scheme Bi-BTE = (Setup, KGen, TagKeys, PosSamp, Enc, GenTag, Vf, Digest, ComputePartKey, CombPartKey, BDec) is IND-CCA secure if for all $\lambda \in \mathbb{N}$, $B \in \mathbb{N}$, $n \in \mathbb{N}$, $t, L \in \mathbb{N}$ such

that $0 < t \leq n$, position $j \in [L]$, and for all $\text{Corr} \subset [n]$ such that $|\text{Corr}| < t$ and if the advantage of any PPT adversaries $\mathcal{A}$ defined by $\text{Adv}_{\text{Bi-BTE}, \mathcal{A}}^{\text{IND-CCA}}$ in $\text{IND-CCA}_{\text{Bi-BTE}, \mathcal{A}}^{\text{Bi-BTE}}$ is negligible.

$$\text{Adv}_{\text{Bi-BTE}, \mathcal{A}}^{\text{IND-CCA}} = \left| \Pr \left[\text{IND-CCA}_{\text{Bi-BTE}, \mathcal{A}}^{\text{Bi-BTE}}(1^\lambda, B, n, t, L, \text{Corr}) = 1 \right] - \frac{1}{2} \right| \leq \text{negl}$$

where the IND-CCA security game $\text{IND-CCA}_{\text{Bi-BTE}, \mathcal{A}}^{\text{Bi-BTE}}$ is as per Fig. 5.

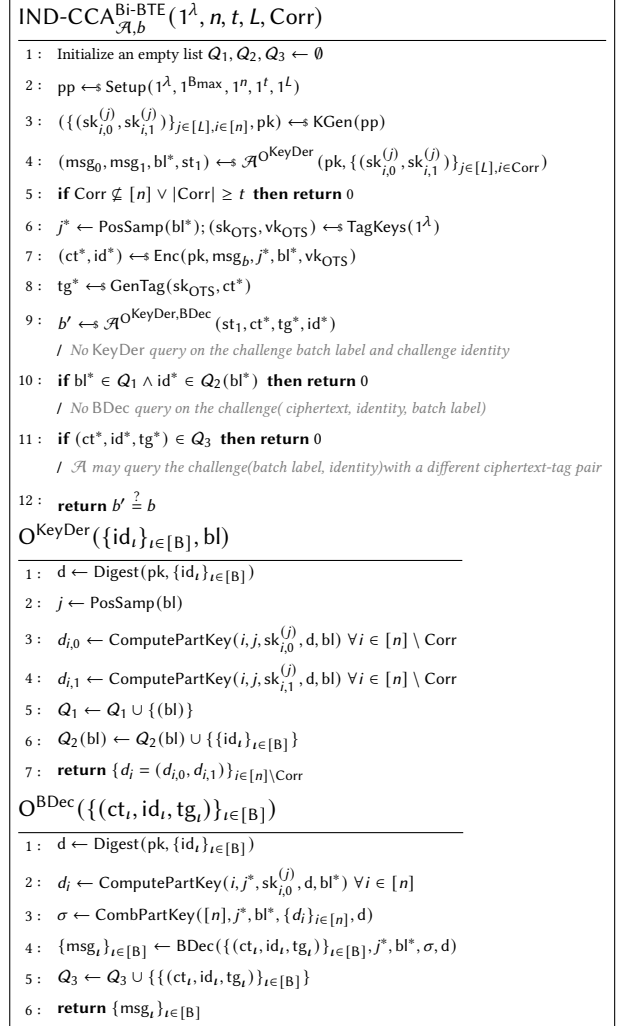


Figure 5: IND-CCA security game for Bi-BTE scheme.

Definition 4.5 (One-Sided Security). A bipartite batched threshold encryption scheme Bi-BTE = (Setup, KGen, TagKeys, PosSamp, Enc, GenTag, Vf, Digest, ComputePartKey, CombPartKey, Dec, BDec) is one-sided secure if for all $\lambda \in \mathbb{N}$, $B \in \mathbb{N}$, $n \in \mathbb{N}$, $t, L \in \mathbb{N}$ such that $0 < t \leq n$, position $j \in [L]$, and for all $\text{Corr} \subset [n]$ such that $|\text{Corr}| < t$ and if the advantage of any PPT adversaries $\mathcal{A}$ defined by $\text{Adv}_{\text{Bi-BTE}, \mathcal{A}}^{\text{OSS}}$ in $\text{OSS}_{\text{Bi-BTE}, \mathcal{A}}^{\text{Bi-BTE}}$ (Fig. 6) is negligible.

$$\text{Adv}_{\text{Bi-BTE}, \mathcal{A}}^{\text{OSS}} = \Pr[\text{OSS}_{\text{Bi-BTE}, \mathcal{A}}^{\text{Bi-BTE}}(1^\lambda) = 1] \leq \frac{1}{2} + \text{negl}$$

4.2 Construction of Bi-BTE Scheme

In this section, we will provide a construction of Bi-BTE scheme using bilinear pairings in prime-order cyclic groups, collision resistant hash functions (Def. A.6) and a one-time signature scheme (Def. A.1). We present our Bi-BTE scheme formally in Fig. 7.

OSS ^{Bi-BTE} _{$\mathcal{A}$} (1^λ)
1 : $pp \leftarrow \text{Setup}(1^\lambda, 1^{\text{Bmax}}, 1^n, 1^t, 1^L)$
2 : $(n, t, L, u, d, \{\text{msg}_{i,0}, \text{msg}_{i,1}\}_{i \in B}, \text{st}) \leftarrow \mathcal{A}(1^\lambda) \quad / \quad t \leq n, u \in [L], d \in \{0, 1\}$
3 : $(\{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{j \in [L], i \in [n]}, \text{pk}) \leftarrow \text{KGen}(pp)$
4 : $(\text{sk}_{\text{OTS}}, \text{vk}_{\text{OTS}}) \leftarrow \text{TagKeys}(1^\lambda); u \leftarrow \text{PosSamp}(\text{bl})$
5 : $(\text{ct}_0 = (\text{ct}_L^{(0)}, \text{ct}_R^{(0)}), \text{id}) \leftarrow \text{Enc}(\text{pk}, \text{msg}_0, u, \text{bl}, \text{vk}_{\text{OTS}})$
6 : $(\text{ct}_1 = (\text{ct}_L^{(1)}, \text{ct}_R^{(1)}), \text{id}) \leftarrow \text{Enc}(\text{pk}, \text{msg}_1, u, \text{bl}, \text{vk}_{\text{OTS}})$
7 : $\text{Sample } b \leftarrow \{0, 1\}$
8 : if $b = 0$ then $\text{ct}^* \leftarrow (\text{ct}_L^{(0)}, \text{ct}_R^{(1)})$
9 : if $b = 1$ then $\text{ct}^* \leftarrow (\text{ct}_L^{(d)}, \text{ct}_R^{(d)})$
10 : $\text{tg}^* \leftarrow \text{GenTag}(\text{sk}_{\text{OTS}}, \text{ct}^*)$
11 : $b' \leftarrow \mathcal{A}(\text{st}, \text{pp}, \text{pk}, \text{ct}^*, \text{tg}^*, \text{id}, \text{msg}_0, \{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{j \in [L] \setminus \{u\}, i \in [n]}, \{\text{sk}_{i,d}^{(u)}\}_{i \in [n]})$
12 : return $b \stackrel{?}{=} b'$

Figure 6: OSS security game for Bi-BTE scheme. If the adversary chooses $d = 0$, the objective is to distinguish between (valid left and valid right) vs. (valid left and invalid right), when only the left secret key at position is available.

Clarifying the notation. As per definition, Bi-BTE ciphertext has two logical component: a left and a right ciphertext. In our construction the ciphertext $\text{ct} = (\text{ct}_1, \text{ct}_2)$ is parsed as follows:

$$\text{ct}_L = \{\text{ct}_1[1], \text{ct}_1[2], \text{ct}_1[3], \text{ct}_2\}, \quad \text{ct}_R = \{\text{ct}_1[4]\}$$

4.3 Theoretical Analysis of Bipartite BTE

In this section, we formally state the theorems associated to the theoretical analysis of our Bi-BTE scheme in Fig. 7.

4.3.1 Bi-BTE Correctness. We now formally state the correctness of our Bi-BTE scheme in Theorem 4.6, which is proven below.

THEOREM 4.6. *The proposed Bi-BTE scheme in Fig. 7 is correct as per Definition 4.2.*

PROOF OF THEOREM 4.6. The CombPartKey algorithm combined all the left and right secret keys into σ^0 and σ^1 , respectively.

$$\begin{aligned} \sigma^0 &= \sum_{i \in \mathcal{J}_0} \left(\lambda_{i,\mathcal{J}} \cdot z_j \cdot s_{i0}^{(j)} (d + H(j, \text{bl})) \right) \\ \sigma^1 &= \sum_{i \in \mathcal{J}_1} \left(\lambda_{i,\mathcal{J}} \cdot y_j \cdot s_{i1}^{(j)} (d + H(j, \text{bl})) \right) \end{aligned}$$

After expanding the first component of the ciphertext:

$$\begin{aligned} \text{ct}_1 &= (\mathbf{r}^T \cdot \mathbf{A}) \\ &= ([r_1 + r_2 \cdot \alpha_j y_j z_j]_2, [r_1(\hat{\omega} - \tau)]_2, -[r_2 \cdot y_j]_2, -[r_2 \cdot z_j]_2) \end{aligned}$$

After pairing with witness vector $\text{ct}_1 \circ \mathbf{w}$, the last two components(combined) derived as follows: $[r_2 \alpha_j y_j z_j (d + H(j, \text{bl}))]_T$. Finally, the decrypted message is given by: $\text{ct}_2 - \text{ct}_1 \circ \mathbf{w}$ as:

$$\begin{aligned} &= -r_2 \left([\alpha_j y_j z_j]_2 \circ H(j, \text{bl}) \right) + r_1 \left([\varphi]_1 \circ [1]_2 \right) + \text{msg} \\ &\quad - (d \circ [r_1]_2) - ([r_2 \cdot \alpha_j y_j z_j]_2 \circ d) - (\pi \circ [r_1(\hat{\omega} - \tau)]_2) \\ &\quad + ([r_2 \alpha_j y_j z_j]_2 \circ d) + ([r_2 \alpha_j y_j z_j]_2 \circ H(j, \text{bl})) \\ &= ([\varphi]_1 \circ [r_1]_2) + \text{msg} - (d \circ [r_1]_2) + (\pi \circ [r_1(\tau - \hat{\omega})]_2) \\ &= \text{msg} \end{aligned}$$

□

4.3.2 Bi-BTE Efficiency. We now formally state the efficiency of our Bi-BTE scheme in Theorem 4.7, which is proven below.

THEOREM 4.7. *The proposed Bi-BTE scheme in Fig. 7 is efficient as per Definition 4.3.*

PROOF. The ComputePartKey and CombPartKey algorithms require $O(1)$ and $O(t)$ group operations resp. (independent of B). □

4.3.3 Bi-BTE Security. We now formally state the security of our Bi-BTE scheme in Theorem 4.8. Due to space constraints, we defer all details of the security proofs of Theorem 4.8 to our full version.

THEOREM 4.8. *The proposed Bi-BTE scheme in Fig. 7 is secure as per Definitions 4.4 and 4.5.*

5 Generic Construction: Bi-BTE to T-BTE

In this section, we provide a construction of our T-BTE scheme using our Bi-BTE scheme and Fingerprinting codes (in Appendix A.5). We present the formal construction of our T-BTE in Fig. 8.

5.1 Theoretical Analysis of Traceable BTE

In this section, we formally state the theorems associated to the theoretical analysis of our T-BTE scheme in Fig. 8.

5.1.1 T-BTE Correctness and Efficiency.

THEOREM 5.1. *The proposed T-BTE scheme in Fig. 8 is correct and efficient as per Definitions 3.2 and 3.3, respectively.*

PROOF OF THEOREM 5.1. Fig. 8 T-BTE's correctness follows easily from that of the underlying Bi-BTE, FC and OTS schemes. The efficiency of T-BTE also follows directly from that of Bi-BTE. □

5.1.2 T-BTE Security.

THEOREM 5.2. *The proposed T-BTE scheme in Fig. 8 is secure as per Definitions 3.4 and 3.5.*

Due to lack of space, we defer proofs of the security theorems for T-BTE to our full version.

5.2 Preventing DOS Attacks via Consistency

Our CCA-secure T-BTE constructions can be extended to ensure consistency (of encryption) by employing an efficient NIZK proof of ciphertext well-formedness. Without this safeguard, attackers could easily flood the mempool, overwhelm validators, and waste blockchain space with malformed transactions. At first glance, this seems to conflict with traceability, which depends on colluding parties inability to detect malformed ciphertexts. However, the tension is resolved by *embedding a simulation trapdoor* of the NIZK into the tracing key, allowing only the tracer to compute simulated proofs. Honest users can publicly verify ciphertext well-formedness, whereas the tracer retains the ability to submit specially crafted ciphertexts for tracing. This approach is discussed in [8, Section 7]. Like all related works, we assume in our threat model that the tracer is trusted *not* to flood the system with simulated proofs.

Setup ($1^\lambda, 1^{B_{\max}}, 1^n, 1^t, 1^L$) <pre> 1: $\mathcal{G} := (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, g_T, \circ) \leftarrow \text{GGen}(1^\lambda)$ 2: Set $\mathcal{M} := \mathbb{G}_T$, $\mathcal{ID} := [p]$, and $\mathcal{T} := \{0, 1\}^\lambda$ 3: Set $H : \mathcal{T} \rightarrow \mathbb{G}_1$, $\text{HOTS} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$, $H_{\text{idx}} : \mathcal{B} \rightarrow [L]$ 4: return $\text{pp} := (\mathcal{G}, H, \text{HOTS})$ </pre>	Vf ($\text{ct}, \text{tg} = (\text{vk}_{\text{OTS}}, \Sigma_{\text{OTS}})$) <pre> 1: return $\text{OTS.Vf}(\text{vk}_{\text{OTS}}, \text{ct}, \Sigma_{\text{OTS}})$ </pre>	CombPartKey ($\mathcal{J}, j, \text{bl}, \{d_i = (b_{i,j}, \sigma_i)\}_{i \in \mathcal{J}}, d$) <pre> 1: $\mathcal{J}_0 := \{i \in \mathcal{J} \mid b_{i,j} = 0\}$ and $\mathcal{J}_1 := \mathcal{J} \setminus \mathcal{J}_0$ 2: Compute $\sigma^{(0)} \leftarrow \sum_{i \in \mathcal{J}_0} \lambda_{0,i,\mathcal{J}} \cdot (\sigma_i)$ 3: Compute $\sigma^{(1)} \leftarrow \sum_{i \in \mathcal{J}_1} \lambda_{0,i,\mathcal{J}} \cdot (\sigma_i)$ 4: return $\sigma := (\sigma^{(0)}, \sigma^{(1)})$ </pre>
KGen (pp) <pre> 1: for $j \in [L]$ do 2: $\alpha_j, y_j, z_j, \tau \leftarrow \mathbb{Z}_p$ 3: $(X_j, Y_j, Z_j) \leftarrow ([\alpha_j y_j z_j]_2, [y_j]_2, [z_j]_2)$ 4: $\{s_i^{(j)}\}_{i \in [n]} \leftarrow \text{SSS.Share}(\alpha_j, n, t)$ 5: for $i \in [n]$ do 6: $\text{sk}_{i,0}^{(j)} \leftarrow z_j \cdot s_i^{(j)}$ and $\text{sk}_{i,1}^{(j)} \leftarrow y_j \cdot s_i^{(j)}$ 7: $\text{pk} := ([\tau]_1, \dots, [\tau^B]_1, [\tau]_2, \{(X_j, Y_j, Z_j)\}_{j \in [L]})$ 8: return $(\{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{j \in [L], i \in [n]}, \text{pk})$ </pre>	Enc ($\text{pk}, \text{msg}, j, \text{bl}, \text{vk}_{\text{OTS}}$) <pre> 1: $\hat{\omega} \leftarrow \{1, \omega, \omega^2, \dots, \omega^{B-1}\}$ / ω is a root of unity in $\mathbb{Z}_p$ of order B. 2: Compute $\varphi := \text{HOTS}(\text{vk}_{\text{OTS}})$ 3: Define $\mathbf{A} \in (\mathbb{G}_2)^{2 \times 4}$ and $\mathbf{b} \in (\mathbb{G}_T)^2$ as: $\mathbf{A} = \begin{pmatrix} [1]_2 & [\hat{\omega} - \tau]_2 & 0 & 0 \\ X_j & 0 & -Y_j & -Z_j \end{pmatrix},$ $\mathbf{b} = \begin{pmatrix} [\varphi]_1 \circ [1]_2 \\ -(X_j \circ H(j, \text{bl})) \end{pmatrix}$ 4: Sample $\mathbf{r} \leftarrow \mathbb{Z}_p^2$ 5: Compute $\text{ct}_1 := \mathbf{r}^\top \cdot \mathbf{A}$, $\text{ct}_2 := \mathbf{r}^\top \cdot \mathbf{b} + \text{msg}$ return $(\text{ct} := (\text{ct}_1, \text{ct}_2), \text{id} := (\hat{\omega}, \varphi))$ </pre>	BDec ($\{(\text{ct}_i, \text{id}_i = (\hat{\omega}_i, \varphi_i), \text{tg}_i)\}_{i \in [B]}, j, \text{bl}, \sigma, d$) <pre> 1: if $\exists i \in [B] : \text{Vf}(\text{ct}_i, \text{tg}_i) = 0$ return $\perp$ 2: parse σ as $(\sigma^{(0)}, \sigma^{(1)})$ 3: $f(P) := \sum_{i \in [0, B]} f_i \cdot P^i$ / B-degree polynomial s.t. $\{f(\hat{\omega}_i) = \varphi_i\} \forall i \in [B]$ 4: for $i \in [B]$ do: 5: $q_i(X) := \sum_{i=0}^B q_i \cdot X^i$ / Compute the opening proof π_i from the poly $q_i(P)$ 6: $q_i(P) := \frac{f(P) - \varphi_i}{P - \hat{\omega}_i}$ 7: $\pi_i := \sum_{i=0}^{B-1} q_i \cdot [\tau^i]_1$ and $\mathbf{w}_i := \begin{pmatrix} d \\ \pi_i \\ \sigma^{(0)} \\ \sigma^{(1)} \end{pmatrix}$ 8: $\text{msg}_i := \text{ct}_{i,2} - \text{ct}_{i,1} \circ \mathbf{w}_i$ 9: return $\{\text{msg}_i\}_{i \in [B]}$ </pre>
TagKeys (1^λ) <pre> 1: return $(\text{sk}_{\text{OTS}}, \text{vk}_{\text{OTS}}) \leftarrow \text{OTS.KGen}(1^\lambda)$ </pre>	Digest ($\text{pk}, \{\text{id}_i = (\hat{\omega}_i, \varphi_i)\}_{i \in [B]}$) <pre> 1: Define $f(P) := \sum_{i \in [0, B]} f_i \cdot P^i$ 2: return $d := \sum_{i \in [0, B]} f_i \cdot [\tau^i]_1$ </pre>	
PosSamp (bl) <pre> 1: return $j \leftarrow H_{\text{idx}}(\text{bl})$ </pre>	ComputePartKey ($i, j, \text{sk}_{i,b}^{(j)}, d, \text{bl}$) <pre> 1: Set $b_{i,j} := b$ / Secret key flag 2: Compute $\sigma_i \leftarrow \text{sk}_{i,b_{i,j}}^{(j)} \cdot (d + H(j, \text{bl}))$ 3: return $d_i := (b_{i,j}, \sigma_i)$ </pre>	
GenTag ($\text{sk}_{\text{OTS}}, \text{ct}$) <pre> 1: $\Sigma_{\text{OTS}} \leftarrow \text{OTS.Sign}(\text{sk}_{\text{OTS}}, \text{ct})$ 2: return $\text{tg} := (\text{vk}_{\text{OTS}}, \Sigma_{\text{OTS}})$ </pre>		

Figure 7: Construction of Bi-BTE scheme.

6 Benchmarks

We implemented a prototype⁴ of our T-BTE scheme in Rust. For this, we ran our benchmarks with different values of the batch size B and the number of parties n with the threshold $t = \lfloor n/2 \rfloor$ and the fingerprinting code [12] lengths $L = a \cdot (c \log c)^2$ where c is the coalition size, i.e., maximum number of user an adversary can corrupt and a is a constant. For our benchmarking, we use $a \in [10, 30]$. We calculated the average times for each algorithms of our T-BTE scheme, over 5 repetitions of the experiment.

Setup. All our experiments are performed on a PC with an Intel(R) Core(TM) i9-14900K, 32GB of DDR4 RAM in single-threaded mode running Linux system.

Curve parameters. We used BLS12-381 pairing-friendly elliptic curve from arkworks framework [4]. In this implementation, each element of $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ takes 96, 192, and 576 bytes respectively. Upon testing, we found the times to run exponentiations in $\mathbb{G}_1$ and $\mathbb{G}_2$ takes an average of 416 and 1352 microseconds respectively, and each pairing evaluation takes 1713 microseconds.

Concrete Parameter Size. The concrete parameter size of our T-BTE scheme as follows: $|\text{pk}| = B \cdot \mathbb{G}_1 + (3L + 1) \cdot \mathbb{G}_2$, $|\text{sk}_i| = 2L \cdot \mathbb{Z}_p$, $|\text{ct}| = 4|\mathbb{G}_2| + |\mathbb{G}_T|$, $|\text{d}| = |\mathbb{G}_1|$, $|\text{d}_i| = |\mathbb{G}_1|$, and $|\sigma| = |\mathbb{G}_1|$.

Experimental Results. We evaluate the concrete performance of T-BTE by answering: (1) runtime of key generation and sizes of pk and all partial sk ; (2) encryption time and ciphertext size; (3) cost of partial batch decryption per server and final batch combination, varying with batch/committee size; (4) message recovery time from

n	t	pk (MB)	sk (MB)	ct (B)	d_i (B)	d (B)
8	4	0.15	0.29	544	57	96
32	16	2	18	544	57	96
64	32	16	24	544	57	96

Table 2: Storage sizes for various user size n with $t = \lfloor n/2 \rfloor$ and $B = 128$. The public-key size grows with both the number of users and the batch size. The secret-key size depends only on the number of users (decryptors). All other sizes remain independent of different user sizes.

B	pk (KB)	sk (MB)	ct (B)	d_i (B)	d (B)
16	722	2	544	57	96
32	724	2	544	57	96
128	738	2	544	57	96
512	792	2	544	57	96

Table 3: Storage sizes for various batch sizes B with user size $n = 16$ and $t = 8$. The public-key size grows with the batch size, albeit with only a very mild dependence. All other sizes remain independent of different batch sizes.

batch-combined values, varying with batch size; and (5) tracing time for a rogue decoder box. We categorize the above questions in terms of storage requirements and runtime performance below.

Storage. The storage requirements are as follows. We calculated the sizes of the public key pk , all partial secret key sk , the ciphertext ct , digest d , and partial batch decryption d_i , for $n \in \{8, 32, 64\}$ with threshold $t = \lfloor n/2 \rfloor$ and fixed batch size $B = 128$ in Table 2 and for different $B \in \{16, 32, 128, 512\}$ with the fixed number of parties $n = 16$ and threshold $t = 8$ in Table 3.

Run Time. Tables 4a to 4c reports the running times of all the algorithms respectively. Tracing time includes the runtime of Enc and that of decryption using rogue decoder D . For our experiments,

⁴Our codebase: <https://github.com/anirban2crypto/traBatchTEnc>

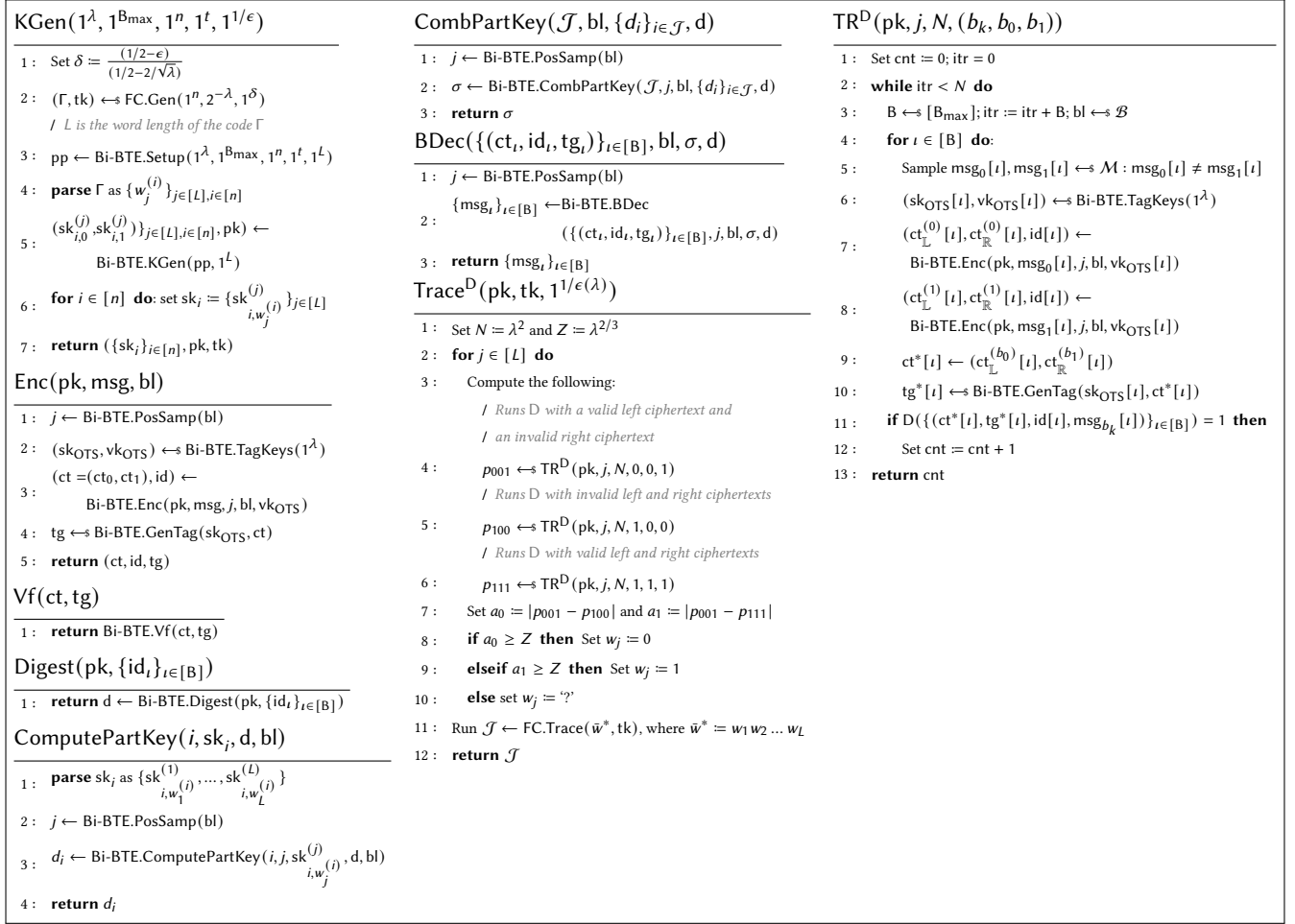


Figure 8: Construction of T-BTE scheme.

n	t	KGen	Enc	ComputePartKey	CombPartKey	BDec
8	4	203.29ms	1.07s	4.30ms	142.44 μ s	417.34ms
16	8	2.67s	1.07s	4.59ms	263.94 μ s	389.35ms
32	16	26.62s	1.07s	5.27ms	536.33 μ s	389.68ms
64	32	602.29s	1.07s	6.74ms	1.23ms	417.48ms

(a) Different n with $t = \lfloor n/2 \rfloor$ and $B = 128$.

B	Trace
4	171.53s
8	339.88s
16	681.57s

(b) $n = 16, L = 2560$.

n	L	Trace
8	480	62.96s
16	2560	339.85s
24	4320	578.34s

(c) $B = 8$

Table 4: Storage and Runtime of T-BTE components.

we implemented a *perfect* decoder with t secret keys hardwired. We tracked its runtime for all different n with fixed size $B = 128$ though current blockchains work with about of 50-100 mempool validators [6]. In Table 5 (Appendix B), we compare the running times of our T-BTE and the BTE from CGPW25 [26], particularly focusing on the total time for a single batch decryption (i.e., total time to decrypt a batch of size $B \in \{16, 32, 128, 512\}$ ciphertexts). Please see Appendix B for more details.

7 Conclusion

In this work, we introduce the first CCA-secure traitor-tracing scheme for threshold encryption supporting batched decryption. We believe this combination of concrete efficiency and a strong security guarantee significantly advances the design of robust accountable threshold encryption systems. An interesting future work is to build a traceable BTE scheme *without* a trusted setup (i.e., with a silent setup) that additionally supports *public* tracing.

ACKNOWLEDGMENTS

We thank the anonymous reviewers of AsiaCCS 2026 for their constructive feedback. The first author was supported by the SERB, Government of India (Grant No. SP/SERB-23-0328). The second author is supported by the “Prime Minister Early Career Research Grant” award from ANRF (via the grant ANRF/ECRG/2024/006105/ENS) under DST, Government of India. The third author is partly supported by the Rishi Khanna Grant and Koita Centre for Digital Health (KCDH) Grant at Ashoka University, and partly supported by the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (grant agreement No 885666, PROCONTRA) while visiting University of Warsaw.

REFERENCES

- [1] Amit Agarwal, Kushal Babel, Sourav Das, and Babak Poorebrahim Gilkalaye. 2025. Time-Lock Encrypted Storage for Blockchains. *Cryptology ePrint Archive*, Report 2025/2048. <https://eprint.iacr.org/2025/2048>
- [2] Amit Agarwal, Rex Fernando, and Benny Pinkas. 2025. Efficiently-Thresholdizable Batched Identity Based Encryption, with Applications. In *CRYPTO 2025, Part III (LNCS, Vol. 16002)*, Yael Tauman Kalai and Seny F. Kamara (Eds.). Springer, Cham, 69–100. https://doi.org/10.1007/978-3-032-01881-6_3
- [3] Ramiro Alvarez and Mehrdad Nojournian. 2020. Comprehensive survey on privacy-preserving protocols for sealed-bid auctions. *Computers & Security* 88 (2020), 101502. <https://doi.org/10.1016/j.cose.2019.03.023>
- [4] arkworks contributors. 2022. arkworks zkSNARK ecosystem. <https://arkworks.rs>
- [5] Kushal Babel, Philip Daian, Mahimna Kelkar, and Ari Juels. 2023. Clockwork Finance: Automated Analysis of Economic Security in Smart Contracts. In *2023 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 2499–2516. <https://doi.org/10.1109/SP46215.2023.10179346>
- [6] Joseph Bebel and Dev Ojha. 2022. Ferveo: Threshold Decryption for Mempool Privacy in BFT networks. *Cryptology ePrint Archive*, Report 2022/898. <https://eprint.iacr.org/2022/898>
- [7] Rob Behnke. 2024. Explained: The Ronin Network Hack. <https://www.halborn.com/blog/post/explained-the-ronin-network-hack-august-2024>
- [8] Rishiraj Bhattacharyya, Jan Bormet, Sebastian Faust, Pratyay Mukherjee, and Hussien Othman. 2025. CCA-Secure Traceable Threshold (ID-based) Encryption and Application. In *ACM CCS 2025*, Chun-Ying Huang, Jyh-Cheng Chen, Shih-Pyng Shieh, David Lie, and Véronique Cortier (Eds.). ACM Press, 2324–2338. <https://doi.org/10.1145/3719027.3744876>
- [9] Dan Boneh. 2023. Cryptography for blockchains: Avoiding common mistakes. https://youtu.be/9yNsOg3_Y3k?si=CihRiox8a4-WwY_H&t=1484
- [10] Dan Boneh, Ran Canetti, Shai Halevi, and Jonathan Katz. 2007. Chosen-Ciphertext Security from Identity-Based Encryption. *SIAM J. Comput.* 36, 5 (2007), 1301–1328. <https://doi.org/10.1137/S009753970544713X>
- [11] Dan Boneh and Matthew K. Franklin. 2001. Identity-Based Encryption from the Weil Pairing. In *CRYPTO 2001 (LNCS, Vol. 2139)*, Joe Kilian (Ed.). Springer, Berlin, Heidelberg, 213–229. https://doi.org/10.1007/3-540-44647-8_13
- [12] Dan Boneh, Aggelos Kiayias, and Hart William Montgomery. 2010. Robust fingerprinting codes: a near optimal construction. In *Proceedings of the 10th ACM Workshop on Digital Rights Management, Chicago, Illinois, USA, October 4, 2010*, Ehab Al-Shaer, Hongxia Jin, and Marc Joye (Eds.). ACM, 3–12. <https://doi.org/10.1145/1866870.1866873>
- [13] Dan Boneh, Aditi Partap, and Lior Rotem. 2024. Accountability for Misbehavior in Threshold Decryption via Threshold Traitor Tracing. In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 317–351. https://doi.org/10.1007/978-3-031-68394-7_11
- [14] Dan Boneh, Amit Sahai, and Brent Waters. 2006. Fully Collusion Resistant Traitor Tracing with Short Ciphertexts and Private Keys. In *EUROCRYPT 2006 (LNCS, Vol. 4004)*, Serge Vaudenay (Ed.). Springer, Berlin, Heidelberg, 573–592. https://doi.org/10.1007/11761679_34
- [15] Dan Boneh and Brent Waters. 2006. A fully collusion resistant broadcast, trace, and revoke system. In *ACM CCS 2006*, Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati (Eds.). ACM Press, 211–220. <https://doi.org/10.1145/1180405.1180432>
- [16] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyang Qu, and Mingyuan Wang. 2025. BEAST-MEV: Batched Threshold Encryption with Silent Setup for MEV prevention. *IACR Cryptol. ePrint Arch.* (2025), 1419. <https://eprint.iacr.org/2025/1419>
- [17] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. 2024. BEAT-MEV: Epochless Approach to Batched Threshold Encryption for MEV Prevention. *Cryptology ePrint Archive*, Report 2024/1533. <https://eprint.iacr.org/2024/1533>
- [18] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. 2025. BEAT-MEV: Epochless Approach to Batched Threshold Encryption for MEV Prevention. In *USENIX Security 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 3457–3476. <https://www.usenix.org/conference/usenixsecurity25/presentation/bormet>
- [19] Jan Bormet, Jonas Hofmann, and Hussien Othman. 2025. Traceable Threshold Encryption without a Trusted Dealer. In *Advances in Cryptology - ASIACRYPT 2025 - 31th International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8–12, 2025, Proceedings, Part III (Lecture Notes in Computer Science)*, Goichiro Hanaoka and Bo-Yin Yang (Eds.). Springer. <https://eprint.iacr.org/2025/342>
- [20] Matteo Campanelli, Bernardo David, Hamidreza Khoshakhlagh, Anders Konring, and Jesper Buus Nielsen. 2022. Encryption to the Future - A Paradigm for Sending Secret Messages to Future (Anonymous) Committees. In *ASIACRYPT 2022, Part III (LNCS, Vol. 13793)*, Shweta Agrawal and Dongdai Lin (Eds.). Springer, Cham, 151–180. https://doi.org/10.1007/978-3-031-22969-5_6
- [21] Ran Canetti, Shai Halevi, and Jonathan Katz. 2004. Chosen-Ciphertext Security from Identity-Based Encryption. In *EUROCRYPT 2004 (LNCS, Vol. 3027)*, Christian Cachin and Jan Camenisch (Eds.). Springer, Berlin, Heidelberg, 207–222. https://doi.org/10.1007/978-3-540-24676-3_13
- [22] Andrea Cerulli, Aisling Connolly, Gregory Neven, Franz-Stefan Preiss, and Victor Shoup. 2023. vetKeys: How a Blockchain Can Keep Many Secrets. *Cryptology ePrint Archive*, Report 2023/616. <https://eprint.iacr.org/2023/616>
- [23] Anirban Chakrabarti, Monosij Maitra, Arup Mondal, and Kushaz Sehgal. 2025. Silent Threshold Traitor Tracing & Enhancing Mempool Privacy. In *ACM CCS 2025*, Chun-Ying Huang, Jyh-Cheng Chen, Shih-Pyng Shieh, David Lie, and Véronique Cortier (Eds.). ACM Press, 1769–1783. <https://doi.org/10.1145/3719027.3765099>
- [24] Benny Chor, Amos Fiat, and Moni Naor. 1994. Tracing Traitors. In *CRYPTO'94 (LNCS, Vol. 839)*, Yvo Desmedt (Ed.). Springer, Berlin, Heidelberg, 257–270. https://doi.org/10.1007/3-540-48658-5_25
- [25] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. 2024. Mempool Privacy via Batched Threshold Encryption: Attacks and Defenses. In *USENIX Security 2024*, Davide Balzarotti and Wenyan Xu (Eds.). USENIX Association. <https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri>
- [26] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. 2025. Practical Mempool Privacy via One-time Setup Batched Threshold Encryption. In *USENIX Security 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 3477–3495. <https://www.usenix.org/conference/usenixsecurity25/presentation/choudhuri>
- [27] Arka Rai Choudhuri, Matthew Green, Abhishek Jain, Gabriel Kaptchuk, and Ian Miers. 2017. Fairness in an Unfair World: Fair Multiparty Computation from Public Bulletin Boards. In *ACM CCS 2017*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, 719–728. <https://doi.org/10.1145/3133956.3134092>
- [28] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2020. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18–21, 2020*, IEEE, 910–927. <https://doi.org/10.1109/SP40000.2020.00040>
- [29] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wöhni. 2023. McFly: Verifiable Encryption to the Future Made Practical. In *FC 2023, Part I (LNCS, Vol. 13950)*, Foteini Baldimtsi and Christian Cachin (Eds.). Springer, Cham, 252–269. https://doi.org/10.1007/978-3-031-47754-6_15
- [30] Nicolas Gailly, Kelsey Melissaris, and Yolan Romailier. 2023. tlock: Practical TimeLock Encryption from Threshold BLS. *Cryptology ePrint Archive*, Report 2023/189. <https://eprint.iacr.org/2023/189>
- [31] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. 2024. Threshold Encryption with Silent Setup. In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 352–386. https://doi.org/10.1007/978-3-031-68394-7_12
- [32] Sanjam Garg, Abhishek Kumarasubramanian, Amit Sahai, and Brent Waters. 2010. Building efficient fully collusion-resilient traitor tracing and revocation schemes. In *ACM CCS 2010*, Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov (Eds.). ACM Press, 121–130. <https://doi.org/10.1145/1866307.1866322>
- [33] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *ASIACRYPT 2010 (LNCS, Vol. 6477)*, Masayuki Abe (Ed.). Springer, Berlin, Heidelberg, 177–194. https://doi.org/10.1007/978-3-642-17373-8_11
- [34] Alireza Kavousi, Duc V. Le, Philipp Jovanovic, and George Danezis. 2023. Blind-Perm: Efficient MEV Mitigation with an Encrypted Mempool and Permutation. *Cryptology ePrint Archive*, Report 2023/1061. <https://eprint.iacr.org/2023/1061>
- [35] Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. 2022. FairBlock: Preventing Blockchain Front-Running with Minimal Overheads. In *Security and Privacy in Communication Networks - 18th EAI International Conference, SecureComm 2022, Virtual Event, October 2022, Proceedings (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Vol. 462)*, Fengjun Li, Kaitai Liang, Zhiqiang Lin, and Sokratis K. Katsikas (Eds.). Springer, 250–271. https://doi.org/10.1007/978-3-031-25538-0_14
- [36] MSN News. 2025. Jurors ask pointed questions in MEV bot trial as deliberations proceed. <https://www.msn.com/en-us/news/crime/jurors-ask-pointed-questions-in-mev-bot-trial-as-deliberations-proceed/ar-AA1PWGET> Accessed: 2025-11-13.
- [37] Claudia Pop, Mirela Prata, Marcel Antal, Tudor Cioara, Ionut Anghel, and Ioan Salomie. 2020. An Ethereum-based implementation of English, Dutch and First-price sealed-bid auctions. In *2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing (ICCP)*, 491–497. <https://doi.org/10.1109/ICCP51029.2020.9266180>
- [38] Kaihua Qin, Liyi Zhou, and Arthur Gervais. 2022. Quantifying Blockchain Extractable Value: How dark is the forest?. In *2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 198–214. <https://doi.org/10.1109/SP46214.2022.9833734>
- [39] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. <https://doi.org/10.1145/359168.359176>

Supplementary Material

A Additional Preliminaries

In this section, we discuss the required technical notations and cryptographic primitives for our proposed scheme.

A.1 Notation

We denote the computational security parameter by $\lambda \in \mathbb{N}$. In threshold settings, we use n to denote the number of parties and t the highest threshold of parties that cannot compromise security. Hence $1 < t \leq n$. We use $[a, b]$ for $a, b \in \mathbb{Z}$, $a \leq b$, to denote $\{a, a+1, \dots, b-1, b\}$. $[b]$ denotes the set $[1, b]$. We denote the concatenation of x and y by $(x||y)$. Given a set $\mathcal{X}$, we denote by $x \leftarrow \mathcal{X}$ the sampling of a value x from the uniform distribution on $\mathcal{X}$. We denote $\text{poly}(\lambda)$ and $\text{negl}(\lambda)$ as any generic (unspecified) polynomial and negligible functions in λ respectively. A function $\text{negl}(\lambda) : \mathbb{N} \rightarrow \mathbb{R}$ is said to be negligible in λ if for every positive polynomial p , $\text{negl}(\lambda) < 1/p(\lambda)$ when λ is sufficiently large. We abbreviate *probabilistic polynomial time* as PPT. We denote by $x = \text{val}$ or $x \leftarrow \text{val}$ the assignment of a value val to the variable x . We denote evaluating a PPT algorithm $\mathcal{A}$ that produces an output out from an input in with randomness $r \leftarrow \{0, 1\}^*$ as $\text{out} \leftarrow \mathcal{A}(\text{in}; r)$, omitting the randomness when it is obvious or not explicitly required. $\mathcal{A}^{\text{O}^{\text{alg}}}$ denotes $\mathcal{A}$ executing with an oracle access to O^{alg} , i.e., $\mathcal{O}$ executes on the inputs of $\mathcal{A}$'s and returns the corresponding outputs.

A.2 Prime-Order Bilinear Groups

We use cyclic groups of prime order p with an asymmetric bilinear map endowed on them. We assume a PPT bilinear group generator algorithm GGen that inputs $\lambda \in \mathbb{N}$ (in unary) and outputs $\mathcal{G} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, g_T, p, \circ) \leftarrow \text{GGen}(1^\lambda)$, where p is a prime of $\Theta(\lambda)$ bits and $\circ : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a non-degenerate, efficiently computable bilinear pairing, where $\mathbb{G}_1, \mathbb{G}_2$ are bilinear group (or pairing group) and $\mathbb{G}_T$ is a multiplicative group, all of prime order p . Let g_1 and g_2 be generators of $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively, and hence $g_T = g_1 \circ g_2$ is a generator of $\mathbb{G}_T$. We express sampling a bilinear group instance as $\mathcal{G} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, [1]_T, p, \circ) \leftarrow \text{GGen}(1^\lambda)$.

We can represent the pairing operation as $[u]_1 \circ [v]_2 = [u]_1 \circ [v]_2 = [u \cdot v]_T$. (This notation makes it easier to write expressions which compose pairings with linear operations.) As a result, the operation is commutative and can be applied to vectors of equal length. For example, $\mathbf{u} := (u_1, \dots, u_n) \in (\mathbb{G}_1)^n$ and $\mathbf{v} := (v_1, \dots, v_n) \in (\mathbb{G}_2)^n$, we have that $\mathbf{u}^T \circ \mathbf{v} = [u_1 \cdot v_1 + \dots + u_n \cdot v_n]_T$. It is further possible to use this notation for matrix-vector multiplication. For example, $\mathbf{A} \in (\mathbb{G}_1)^{n \times m}$ and $\mathbf{b} \in (\mathbb{G}_1)^m$ then $\mathbf{A} \circ \mathbf{b}$ is the vector in $(\mathbb{G}_T)^n$ with the coordinates $(\mathbf{A}_1 \circ \mathbf{b}, \dots, \mathbf{A}_n \circ \mathbf{b})$ where $\mathbf{A}_i$ is the i -th row vector of the matrix $\mathbf{A}$.

A.3 One-Time Signature Schemes

Definition A.1 (One-Time Signature). A one-time signature schemes OTS is a tuple of PPT algorithms $(\text{KGen}, \text{Sign}, \text{Vf})$ which have the following syntax:

- (1) $(\text{vk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$: The randomized key generation algorithm takes as input the security parameter λ and outputs the verification-secret key-pair (vk, sk) .
- (2) $\sigma \leftarrow \text{Sign}(\text{sk}, \text{msg})$: The signing algorithm takes an input a secret key sk and a message $\text{msg} \in \{0, 1\}^*$, and outputs a signature σ . The signature σ allows it to be proven later that the message belongs to the owner of the key.
- (3) $1/0 \leftarrow \text{Vf}(\text{vk}, \text{msg}, \sigma)$: The verification algorithm outputs 1 if σ is a valid signature (was generated via sk) on msg under the verification key vk , and outputs 0 otherwise.

A one-time signature scheme OTS must satisfy the standard notions of **correctness** (Definition A.2) and **unforgeability** (Definition A.3) (i.e., existentially unforgeable against adaptive chosen message attacks (EUF-CMA)) described below.

Definition A.2 (Correctness). A one-time signature scheme $\text{Sig} = (\text{KGen}, \text{Sign}, \text{Vf})$ is correct if for any security parameter λ and any message $\text{msg} \in \{0, 1\}^\lambda$, we have that

$$\Pr \left[\text{Vf}(\text{vk}, \text{msg}, \sigma) = 1 \mid \begin{array}{l} (\text{vk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda) \\ \sigma \leftarrow \text{Sign}(\text{vk}, \text{sk}, \text{msg}) \end{array} \right] = 1$$

Definition A.3 (Unforgeability). A one-time signature scheme $\text{OTS} = (\text{KGen}, \text{Sign}, \text{Vf})$ is unforgeable if for any security parameter λ and for all PPT adversaries $\mathcal{A}$, advantage $\text{Adv}_{\text{OTS}, \mathcal{A}}^{\text{unForge}}$ of the $\text{unForge}_{\text{OTS}}^{\mathcal{A}}$ (in Figure 9) is negligible.

$$\text{Adv}_{\text{OTS}, \mathcal{A}}^{\text{unForge}} = \Pr[\text{unForge}_{\text{OTS}}^{\mathcal{A}} = 1] \leq \text{negl}(\lambda)$$

$\text{unForge}_{\text{OTS}}^{\mathcal{A}}(1^\lambda)$
1: Initialize an empty list $Q \leftarrow \emptyset$
2: $(\text{vk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$
3: $(\text{msg}', \sigma') \leftarrow \mathcal{A}^{\text{OSign}}(1^\lambda, \text{vk})$
4: if $\text{Vf}(\text{vk}, \text{msg}', \sigma') = 1 \wedge \text{msg}' \notin Q$
5: return 1
6: else return 0
$\text{OSig}(\text{msg})$
1: $Q \leftarrow Q \cup \text{msg}$
2: $\sigma \leftarrow \text{Sign}(\text{vk}, \text{sk}, \text{msg})$
3: return σ

Figure 9: Unforgeability Game for OTS scheme.

A.4 Shamir Secret Sharing [39]

Let $P(x) = c_0 + c_1x + \dots + c_tx^t$ be a polynomial of degree t , where each coefficient $c_i \in \mathbb{F}$ for some finite field $\mathbb{F}$. Observe that for any $\alpha \in \mathbb{F}$ and any set $\mathcal{J} = \{\beta_1, \dots, \beta_\ell\} \in \mathbb{F}^\ell$ such that $\ell \geq (t+1)$, there exists a set of scalars $\{\lambda_{\alpha, j, \mathcal{J}}\}_{j \in [\ell]}$ such that $P(\alpha) = \sum_{j \in [\ell]} \lambda_{\alpha, j, \mathcal{J}} P(\beta_j)$. Each scalar, called the Lagrange coefficient $\lambda_{\alpha, j, \mathcal{J}} = \prod_{j' \in \mathcal{J}, j' \neq j} \frac{x - x_{j'}}{\alpha - x_{j'}}$ depends *only* on the set $\mathcal{J}$ and the evaluation point α . Given the above observation, we now recall the Shamir Secret Sharing scheme [39]. Let p be a prime, and n, t be positive integers such that $t < n$. A (p, n, t) -Shamir Secret Sharing $((p, n, t)\text{-SSS for short})$ scheme is a pair of algorithms $(\text{Share}, \text{Recon})$ described below.

Security. Any (p, n, t) -SSS scheme provides the following (information-theoretic) security guarantee: for any uniformly random secret $s \leftarrow \mathbb{Z}_p$ such that $(s_1, \dots, s_n) \leftarrow \text{Share}(s, p, n, t)$, given any subset of the shares $\mathcal{J} \subset (s_1, \dots, s_n)$ such that $|\mathcal{J}| \leq t$, s remains distributed uniformly at random.

Share(s, p, n, t): On input $s \in \mathbb{Z}_p$, proceed as follows:

- 1: Sample $c_1, \dots, c_t \leftarrow \mathbb{Z}_p$
- 2: Let $P(x) = s + c_1x + \dots + c_tx^t$
- 3: **for** $i \in [n]$ **do**: Compute $s_i = P(i) \in \mathbb{Z}_p$
- 4: **return** $(s_1, \dots, s_n)$

Recon($s_{j_1}, \dots, s_{j_\ell}$): On input $(s_{j_1}, \dots, s_{j_\ell}) \in \mathbb{Z}_p$, proceed as follows:

- 1: **if** $\ell < (t + 1)$, **then return** $\perp$.
- 2: **else** $s = P(0) = \sum_{k \in [\ell]} \lambda_{0, j_k} s_{j_k}$
/ using the Lagrange coefficients $\{\lambda_{0, j_k}\}_{k \in [\ell]}$
- 3: **return** s

Figure 10: (p, n, t) -Shamir Secret Sharing scheme [39]

A.5 Collusion Resistant Fingerprinting Codes

Here, we recall the notion of Finger printing codes [12, 13].

Definition A.4 (Collusion Resistant Fingerprinting Codes). A collusion-resistant fingerprinting codes FC consists of the following tuple of PPT algorithms (Gen, Trace) with the following syntax:

- $(\Gamma, \text{tk}) \leftarrow \text{Gen}(1^\epsilon, 1^N, 1^\delta)$: The probabilistic code generation algorithm takes as input a security parameter ϵ , a number $N > 0$ of words, and a noise bound δ , codes capable of tracing words even when the pirate adversarially destroys a δ fraction of the marks in the fingerprint. It outputs a code Γ , which is a set of N words in $\{0, 1\}^L$, and a tracing key tk . The length L of the code words is a function $L = L(\epsilon, N, \delta)$ of the code size N , the security parameter ϵ , and the noise bound δ .
- $\mathcal{J} \leftarrow \text{Trace}(\bar{w}^*, \text{tk})$: The deterministic tracing algorithm takes as input a noisy word $\bar{w}^*$ and the tracing key tk , and outputs a subset $\mathcal{J} \subseteq [L]$.

Security. The security property of fingerprinting codes says that if an adversary in possession of a subset $W \subseteq \Gamma$ of codewords generates a δ -noisy word $\bar{w}^*$ which is feasible for W (i.e., $\bar{w}^* \in F_\delta(W)$), then $\mathcal{J} \leftarrow \text{Trace}(\bar{w}^*, \text{tk})$ outputs a set $\mathcal{J}$ that corresponds to a non-empty subset of W .

Definition A.5 (Fully Collusion Resistant). A fingerprinting codes FC = (Gen, Trace) is fully collusion resistant if for all PPT adversaries $\mathcal{A}$, for all $N > 0$, $\epsilon \in (0, 1]$, $\delta \in [0, 1]$ and for all subsets $\mathcal{I} \subseteq [N]$, it holds that:

$$\text{Adv}_{\text{FC}, \mathcal{A}}^{\text{CR}}(\lambda) = \Pr[\text{CR}_{\text{FC}}^{\mathcal{A}}(N, \mathcal{I}, \epsilon, \delta) = 1] < \epsilon$$

where the fully collusion resistant security game $\text{CR}_{\text{FC}}^{\mathcal{A}}(N, \mathcal{I}, \epsilon, \delta)$ is defined in Fig. 11.

Known Constructions. The fingerprinting codes we use, that allow for tracing even in the presence of a bounded fraction of unknown bits (represented by ‘?’ symbols), are called *robust* fingerprinting codes. Specifically, in our implementation study, we

$\text{CR}_{\text{FC}}^{\mathcal{A}}(N, \mathcal{I}, \epsilon, \delta)$

- 1: $(\Gamma, \text{tk}) \leftarrow \text{Gen}(1^\epsilon, 1^N, 1^\delta)$ / $\Gamma = \{\bar{w}^{(1)}, \dots, \bar{w}^{(N)}\}$
- 2: $W \leftarrow \{\bar{w}^{(i)}\}_{i \in \mathcal{I}}$
- 3: $\bar{w}^* \leftarrow \mathcal{A}(W)$
- 4: **if** $\bar{w}^* \notin F_\delta(W)$ **then return** 0
- 5: $\mathcal{J} \leftarrow \text{Trace}(\bar{w}^*, \text{tk})$
- 6: **if** $\mathcal{J} = \emptyset \vee \mathcal{J} \not\subseteq \mathcal{I}$ **then return** 1
- 7: **else return** 0

Figure 11: Fully Collusion Resistant Security Game for FC scheme.

use the fingerprinting codes construction by Boneh, Kiayias, and Montgomery [12].

A.6 Hash Function

Here, we recall the syntax and security definition for the hash function below.

Definition A.6 (Hash Function). A hash function is a deterministic algorithm $H : \{0, 1\}^* \rightarrow \{0, 1\}^N$ that maps an input of arbitrary length to a fixed-length output of N bits.

Definition A.7 (Collision Resistance). Let H be a hash function and it is infeasible to find any two distinct inputs x, x' such that: $H(x) = H(x')$, formally,

$$\text{Adv}_{H, \mathcal{A}}^{\text{CR}} = \Pr[(x, x') \leftarrow \mathcal{A} : x \neq x' \wedge H(x) = H(x')] \leq \text{negl}$$

B Additional Experimental Results

In Table 5, we present a runtime comparison between our T-BTE scheme and CGPW25 [26], showing the total time required for a single batch decryption (i.e., the total time to decrypt a batch of $B \in \{16, 32, 128, 512\}$ ciphertexts). The time measurements for CGPW25 are taken from [26, Table 2].

Scheme	B	ComputePartKey	CombPartKey + BDec
CGPW25 [26]	16	49.2ms	60.5ms
Our T-BTE	16	1.45ms	$257.16\mu\text{s} + 43.37\text{ms}$
CGPW25 [26]	32	199.8ms	168.1ms
Our T-BTE	32	2.04ms	$259.88\mu\text{s} + 90.18\text{ms}$
CGPW25 [26]	128	809.9ms	646.1ms
Our T-BTE	128	4.59ms	$263.94\mu\text{s} + 389.35\text{ms}$
CGPW25 [26]	512	3203.9ms	3026.5ms
Our T-BTE	512	13.46ms	$274.57\mu\text{s} + 1.67\text{s}$

Table 5: Runtime comparison with [26] for different batch size B with users $n = 16$ and threshold $t = 8$.

We are not able to compare our results with the other two most relevant works. In [2], the source code and key parameter details, including the user size used in their experiments, are not available. Likewise, [8] does not provide an implementation of their bipartite threshold encryption construction, preventing an empirical comparison.